A DISSERTATION SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF MASTER OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2025

Department of Computer Science

Contents

Word Count: 8946 (source: OverLeaf)

List of Tables

List of Figures

Abstract

ssertat on su m tte to T e Un vers ty o Manc ester
for the degree of Master of Science, 2025

Neural networks have been shown to achieve high accuracy scores across various tasks, however, they lack deterministic robustness guarantees. This leaves them vulnerable to small, adversarial input perturbations that can change the output drastically. This issue has serious consequences for safety-critical systems, such as self-driving cars and medical diagnostics. Existing approaches include Lipschitz-bounded and adversarially trained networks: Lipschitz-bounded networks guarantee robustness, while sacrificing accuracy, whereas adversarial training is capable of high accuracy but cannot guarantee robustness. Leaving open the question: *Is it possible to both guarantee robustness and maintain high accuracy?* We propose a solution that enforces guaranteed robustness only around high-confidence predictions by dynamically adjusting the Lipschitz constant based on the output confidence of a neural network. This allows the network to maintain a high degree of flexibility around the decision boundary. Experiments on synthetic data show that this method matches existing approaches in accuracy, while guaranteeing robustness, showing that confidence-based approaches are a viable pathway to robustness.

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

Chapter 1

Introduction

1.1 Motivation

Neural networks have risen in popularity in recent years due to their ability to approximate data-generating functions well, given sufficient examples. This is enabled by the increasing amount of compute available, efficient training algorithms, and the vast hypothesis space of these networks [HSW89]. As such, there are very few guaranteed properties that we can attribute to any fitted neural network function in general. A property of interest within the field of AI safety is robustness. For self-driving cars, robustness can provide protection against sensor malfunctions, weather changes, occlusions, and malicious attacks. For example, [EEF+18] demonstrated a targeted physical attack that causes models to misclassify road signs. Some methods are capable of disabling object detection systems by including a specific patch in the image [TRG19]. In addition, some methods are capable of overcoming face-recognition systems and impersonate other people by wearing special engineered glasses [SBBR16]. In medical diagnostics, adversarial examples have been shown to compromise deep learning models used for fundoscopy, chest X-rays, and dermoscopy, where small perturbations to medical images can lead to misdiagnosis with potentially life-threatening implications [MNG+21].

The implications of adversarial examples extend beyond technical considerations, namely to the societal domain. As autonomous systems are used more commonly in transportation, healthcare, security, and other everyday applications, ensuring robustness against adversarial perturbations becomes an ethical requirement for maintaining public trust and safety. Without robustness, the consumer of an AI system can never be fully convinced that the system will not malfunction in critical decision-making moments. Requirements focused on adversarial robustness for high-risk AI systems are already implemented in Article 15 of the EU AI Act (2024) [PC24].

There exist methods in the literature that exploit the lack of robustness optimization

within the training objectives by producing adversarial examples. Such as the Fast Gradient Sign Method (FGSM) [GSS15], which perturbs an input data point in the direction of most change with respect to the network output, or the Projected Gradient Descent (PGD) method [MMS+18]. These perturbations can often be of imperceptible size to a human observer. See figure 1.1 for an illustration of imperceivable perturbations generated by the PGD method on the MNIST dataset [LBBH98]. The figure shows how small pixel perturbations on an image can flip the class in a binary classification setting and produce a misclassified output with high confidence. The adversarial example produced is almost indistinguishable from the original example, showcasing the potential danger to AI systems if robustness is ignored. Critically, adversarial examples often are transferable across different architectures and training procedures, suggesting that this vulnerability is intrinsic to neural networks rather than an artifact of specific implementations [SZS+14]. Other forms of attack include sticker attacks [BMR+17], where the authors created a universal, robust, targeted, physical image patch that enables real-time misclassification when placed in a scene.
In addition to safety-critical systems, robustness has also been shown to relate to generalization performance [XM12, TPV25], hence it is also of theoretical importance for model performance.

More specifically, a neural network is called **robust** if changing the input by a small perturbation does not change the output classification drastically. I.e., the output class remains fixed under perturbations of some size and under some additional conditions. In the case that this robustness property is guaranteed to hold only for confident outputs, we call the neural network robust around high confidence regions only. Robustness is more formally defined in section 2.1.2. This project mainly concerns the latter type of robustness due to the limitations of the former, as discussed in the following sections.

1.2 Existing solutions

Robustness approaches can often be categorized based on how much they constrain the learning process. Namely, whether they impose hard constraints or soft constraints.

1.2.1 Soft constraints

These are approaches that only incentivize robust solutions, however, they have very few mathematical guarantees. For example, Adversarial Training and Sobolev Training.
Adversarial training utilizes adversarial attacks, such as FSGM [GSS15] and PGD [MMS+18] during training. Given an attack model, the training objective is modified by implicitly

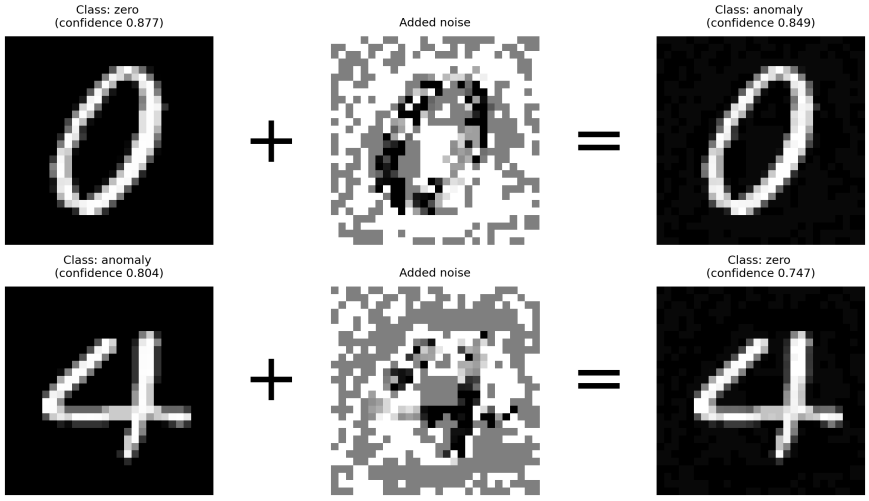


Figure 1.1: A model trained to classify zeros and anomalies in the MNIST dataset has been perturbed using the Projected Gradient Descent (PGD) adversarial attack. The original inputs are shown on the left with the respective predicted labels and confidences. The perturbation noise found by PGD is shown in the middle. The resulting adversarial examples are shown on the right alongside the new predicted class and the corresponding confidence. Both images look nearly identical, however, are classified differently with high confidence.

adding adversarial examples with the original labels to the training set. Importantly, robustness is only promoted for points in the training set, and there are no deterministic guarantees [ZCS+19]. The resulting modified loss function has the following form for a training point $(\vec{x}_i, y_i)$:

$$\mathcal{L}_{\text{Adversarial}}(f_\theta, \vec{x}_i, y_i) = \alpha \mathcal{L}(f_\theta(\vec{x}_i), y_i) + (1-\alpha)\mathcal{L}(f_\theta(\vec{x}'_i), y_i) \tag{1.1}$$

Where $\vec{x}'_i$ is the adversarial example found by some attack, f_θ is the model, $\mathcal{L}$ is some loss function, and $\alpha \in [0,1]$ controls how much weight is given to the adversarial example. This formulation of adversarial training is used as a baseline for our comparative study.

Sobolev training is a method of modifying the loss function to also support learning of function derivatives with respect to input [COJ+17]. This can be seen as a soft version of Lipschitz-bounded networks, which also achieve robustness by limiting the rate of change. For a training point $(\vec{x}_i, y_i)$, the modified loss function can be written as:

$$\mathcal{L}_{\text{Sobolev}}(f_\theta, \vec{x}_i, y_i) = \mathcal{L}\big(f_\theta(\vec{x}_i), y_i\big) + \lambda(J_{f_\theta}(\vec{x}_i), J_y(\vec{x}_i)) \tag{1.2}$$

Where J_{f_θ} is the Jacobian of f_θ and J_y captures some target Jacobian properties, and λ is some loss function that measures how well J_{f_θ} satisfies these properties.

1.2.2 Hard constraints

These are approaches that mathematically guarantee robustness. These guarantees may be deterministic or probabilistic. For example, Lipschitz-bounded networks and Randomized Smoothing.

A prominent class of hard constraint solutions for robustness is Lipschitz-bounded neural networks [ACB17, CBG+17]. Lipschitz continuity is a concept named after the 19th-century German mathematician Rudolf Lipschitz. It provides a measure of how rapidly a function can change, complementing the concept of derivatives. A Lipschitz-continuous function guarantees that the difference in function values is proportionally bounded by the distance between inputs. This concept, in recent years, has been adopted as a way to implement robustness in neural networks. Namely, a Lipschitz-bounded network achieves robustness by bounding the rate of change of the network with respect to the input globally. This guarantees that the output under perturbations will not deviate much from the original output. More formally, if $f(\vec{x})$ is an L-Lipschitz network, then for all inputs $\vec{x}, \vec{x}'$ we have that $d(f(\vec{x}), f(\vec{x}')) \leq Ld(\vec{x}, \vec{x}')$ for some distance metric d (for example, the L_2 norm). Therefore, given a sufficiently small L, all points close in input space produce very similar outputs. Popular techniques to enforce this constraint include spectral normalization [MKKY18], and architectural design choices [PBBL24, LHA+19, PL22], such as orthogonalization of linear layers. Lipschitz networks serve as the theoretical foundation for many certified robustness methods as they enable mathematical guarantees of robustness due to the constraints they place on the rate of change [TSS18]. However, enforcing tight Lipschitz bounds often involves a trade-off between robustness and model capacity, which remains an active research challenge. This project is heavily based on Lipschitz theory, thus Lipschitz continuity and neural networks are formally defined and discussed in Section 2.1.

Another class of hard-constraint robustness methods is based on randomized smoothing, which constructs a smoothed classifier by averaging predictions over random noise added to inputs [CRK19]. Importantly, this approach ensures robustness guarantees with probabilistic and not deterministic bounds on the classifier's output within a perturbation radius.

In this project, we only consider Lipschitz-bounded networks and adversarially trained networks for a comparative study against our proposed method. These were chosen as they are representative of the two common paradigms (hard-constraint and soft-constraint, respectively). Lipschitz-bounded networks were a natural choice since our method directly utilizes Lipschitz bounds. Adversarial training is a widely used and foundational baseline, making it useful for comparison.

1.3 Research Challenge

Soft-constraint-based candidate solutions, such as Adversarial or Sobolev training, are capable of achieving good accuracy and robustness. These approaches do not suffer from issues related to flexibility, as no hard constraints are applied to the architecture. Hence, the hypothesis space remains unchanged. However, soft-constraint-based solutions are not suitable for safety-critical applications as they do not provide deterministic robustness guarantees. Similarly, Randomized Smoothing cannot provide deterministic guarantees - only probabilistic.

Lipschitz-bounded networks have been shown to provide deterministic robustness guarantees that are confidence/margin[1]-dependent, however, due to the strict Lipschitz bound requirement often sacrifice their expressive capabilities (the network cannot exceed the Lipschitz bound). As a result, these models may struggle to learn functions with varying gradient magnitudes across the input space because the global Lipschitz bound enforces uniform smoothness. Namely, it has been shown that simple functions, such as the absolute value function, cannot be approximated by norm-constrained ReLU networks, which restrict their hypothesis space significantly [ALG19].
For an illustration of the heavy constraint Lipschitz continuity places on the hypothesis space, see figure 1.2. This figure illustrates how a nearly piecewise-linear function with a smooth transition (simulated using the arctan function) contains large variability in derivative magnitude. If a Lipschitz-bounded model were to be fit to the function, it would either have to have a large Lipschitz constant, which would provide fewer guarantees, or a small Lipschitz constant, which would not be able to capture the ”jump”. This illustrates the robustness-accuracy trade-off that is often observed in the Lipschitz-bounded network literature. However, some theoretical results show this trade-off is not necessitated by the Lipschitz bounds. Namely, for separable datasets, there exist local Lipschitz constants that can ensure both robustness and accuracy [YRZ+20]. However, this result is only theoretical, while the trade-off is often observed empirically.

The global Lipschitz constant bound is useful in robustness proofs because it allows one to reason about the distance from the decision boundary given a data point. Therefore, statements about robustness can be made, which often depend on the margin, i.e., how close the other class is in terms of logits. Given a small margin, Lipschitz-bounded networks can only provide a very weak bound. I.e., only very small perturbations are guaranteed not to change the classification. Additionally, the model is already uncertain about the prediction. Therefore, it may not be appropriate to ensure robustness for points near the decision boundary. To the best of our knowledge, there exists a research gap for methods that only require robustness when the margin is large enough, i.e., robustness around high confidence regions

[1] A margin in the Lipschitz Neural Network literature is the difference between the highest and second highest absolute logit in multi-class classification. It can be viewed as a measure of confidence.

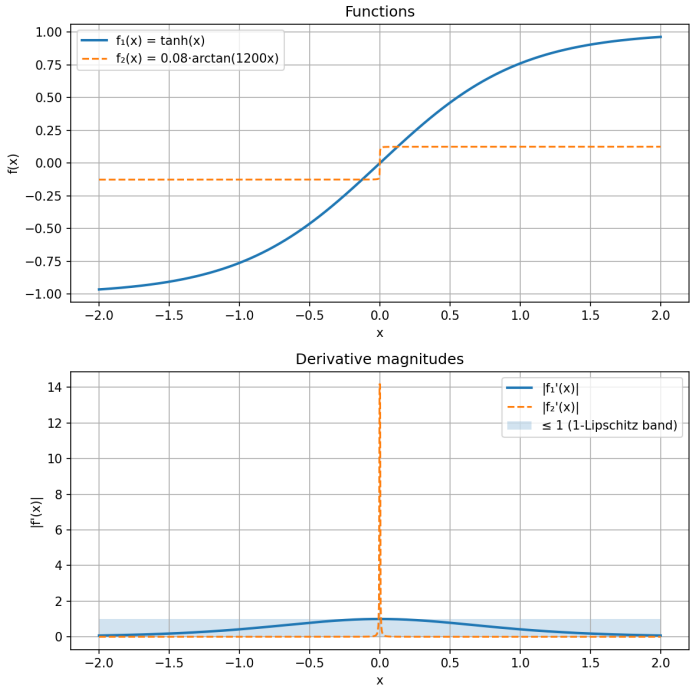


Figure 1.2: The plot contains two functions - a hyperbolic tangent function (blue, 1-Lipschitz) and an inverse tangent function (orange, 0-Lipschitz almost everywhere but near $x = 0$). This illustrates that derivative magnitudes for fairly simple functions can be very variable across the input space, which tightly bounded Lipschitz networks might struggle to capture.

only. While there exist approaches that can verify global robustness around high confidence regions [ABC+24], there do not exist approaches in the literature that deterministically guarantee such a property, while preserving model accuracy.

1.4 Project Requirements

The high-level goal of the project is a novel solution that overcomes the accuracy-robustness trade-off empirically observed in Lipschitz-bounded network literature. The aim is to achieve this by ensuring robustness only around high confidence regions, taking inspiration from the authors of [ABC+24]. The following is a full list of project requirements:

1. The proposed method must apply to a multi-class classification problem with multi-dimensional inputs.

2. A trivial solution to the problem is to guarantee that the outputs are near the decision boundary (confidence always near 0.5 in the binary case) and the function never enters high-confidence regions. Therefore, it is required that the method is capable of finding solutions with non-zero high-confidence coverage (i.e., the number of test points assigned high confidence is non-zero).

3. The solution must be parametrized by the confidence threshold κ and the radius of robustness ε. Specifically, there must exist mathematical guarantees that for inputs classified with a confidence over κ, any perturbation in an ε-radius does not change the classification.

4. The solution must improve upon existing Lipschitz-bounded neural networks by overcoming the accuracy-robustness trade-off. Namely, the novel solution should be robust and empirically demonstrate higher accuracy than Lipschitz-bounded networks.

5. At least two novel methods should be created that satisfy requirements (1)-(4).

However, due to time-based and other restrictions throughout the project, not all requirements were completed. Mainly, only one method was evaluated (requirement 5), and this method is only capable of modelling binary classification tasks (requirement 1). In addition, the capabilities of the model were only tested on a very simple dataset, hence no conclusions on meeting requirement 4 can be made.

1.5 Solution

In this project, we propose a new architecture that partially meets all requirements above. Our solution hypothesizes that relaxation of tight Lipschitz bounds to only meaningful parts of the input space, i.e., high-confidence regions, would expand the hypothesis space enough to enable high-accuracy solutions, while also enabling guarantees of robustness around these high-confidence regions.

The proposed method achieves this by dynamically adjusting its local Lipschitz constant depending on the final confidence. It consists of a Lipschitz neural network g (or any Lipschitz-bounded function/model), representing the initial confidence, and a function λ that maps the initial confidence to a local Lipschitz constant. Namely, define a function $h_\theta(y) = y\lambda_\theta(y)$ such that the functional form of the model is $f_\theta(\vec{x}) = h_\theta(g_\theta(\vec{x})) = g_\theta(\vec{x})\lambda_\theta(g_\theta(\vec{x}))$. Importantly, the definition of λ depends on the confidence threshold and the robustness radius, which, therefore, provides deterministic guarantees of robustness in this setup. See Figure 1.3 for a block diagram of the network architecture.

A dynamic Lipschitz constant allows the network to use different rates of change around inputs classified with high confidence and ones with low confidence. For example, when the network is highly confident, it may be desirable for the rate of change to be low to guarantee robustness properties in the neighbourhood. However, unconfident outputs indicate closeness to the decision boundary, where we do not necessarily require a low rate of change. This demonstrates the different local Lipschitz constant requirements depending on output confidence, which global Lipschitz networks do not support. The proposed method allows

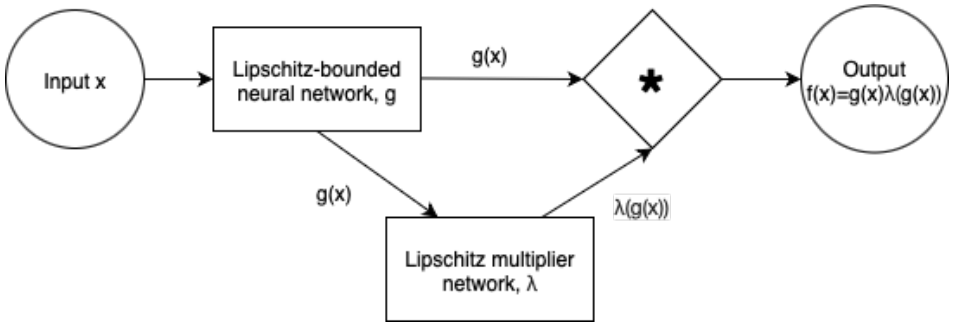


Figure 1.3: Diagram of the proposed network architecture

the model to dynamically adapt to the requirements of the specific domain region.
We hypothesize that our approach allows enough flexibility with respect to the local rate of change to overcome the decrease in model expressivity that is empirically observed with Lipschitz-bounded models.

1.6 Results

We mathematically prove that our method achieves robustness around high confidence predictions in the binary classification case for multi-dimensional inputs. We produce sufficient conditions for robustness, and show that our method meets them. In addition to theoretical results, we also conduct two empirical studies on synthetic data generated by the sigmoid function. Firstly, we show that the method matches existing approaches with respect to accuracy, while also guaranteeing robustness. This serves as proof of concept that confidence-based dynamic Lipschitz constant approaches are a viable pathway to robustness. Second, we investigate the effects of hyperparameters of this method (discussed in 2.2.2) and show that the method is mostly sensitive only to the Lipschitz constant of the initial logit network, g.

Chapter 2

Methodology

This section introduces the preliminaries, notation, and the proposed method alongside proofs of global robustness around high-confidence predictions.
While Lipschitz continuity in neural networks is also often studied with respect to the L_∞ norm, the focus of this paper will be exclusively on the L_2 norm. Even though both are equivalent when the input is unidimensional, it proved to be more natural to extend proofs to multidimensional inputs under the L_2 metric. Many of the results can be extended to the L_∞ norm in future work. The L_2 norm for any vectors $\vec{x}, \vec{z} \in \mathbb{R}^d$ is defined as

$$||\vec{x}-\vec{z}||_2 = ((\vec{x}-\vec{z})^T(\vec{x}-\vec{z}))^{\frac{1}{2}} = (\sum_{i=1}^{d}(x_i-z_i)^2)^{\frac{1}{2}} \tag{2.1}$$

To guide readers without an in-depth knowledge of the field or mathematics, a non-technical explanation in the form of a remark will be presented at various points throughout the section, for example, after each proof.

2.1 Notation and preliminaries

This section introduces crucial definitions, such as Lipschitz continuity, and required knowledge, such as neural networks and approaches to bound their Lipschitz constant.

2.1.1 Lipschitz continuity

Definition 2.1.1 (Global Lipschitz Continuity). *A global Lipschitz constant of a function $f : \mathbb{R}^{d_0} \to \mathbb{R}^{d_1}$ is defined as any $l \in [0, \infty]$ such that,*

$$L(f) := \sup_{\vec{x},\vec{z}\in\mathbb{R}^{d_0}} \{\frac{||f(\vec{x})-f(\vec{z})||_2}{||\vec{x}-\vec{z}||_2}\} \leq l \tag{2.2}$$

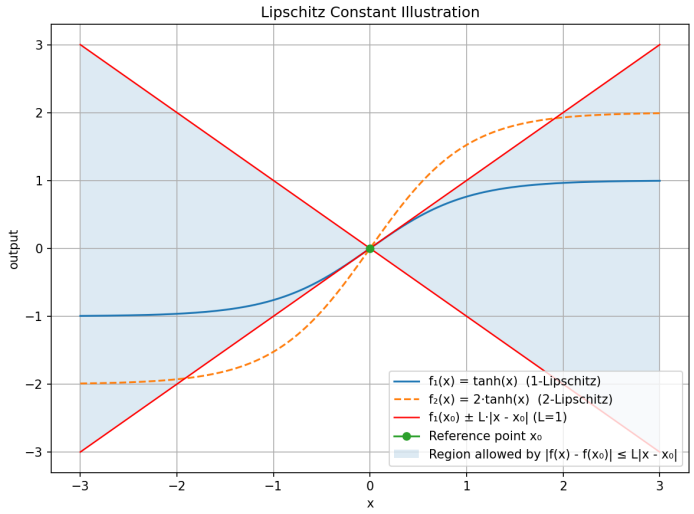


Figure 2.1: The plot contains a hyperbolic tangent function (blue, 1-Lipschitz), and a scaled hyperbolic tangent function (orange, 2-Lipschitz). Given a fixed point, $x_0 = 0$, the shaded region captures a bound on function values that satisfy 1-Lipschitz continuity. As the scaled tanh function is outside of this region, it is not 1-Lipschitz continuous.

Where $L(f)$ is defined to be the minimal Lipschitz constant for the function f. Equivalently, one can say that the function f is l-Lipschitz continuous.

Definition 2.1.2 (Local Lipschitz Continuity). *A local Lipschitz constant $l \in [0,\infty]$ for a function $f : \mathbb{R}^{d_0} \to \mathbb{R}^{d_1}$ at a given input, $\vec{z} \in \mathbb{R}^{d_0}$ that holds for all points in an ε-radius around is defined as follows,*

$$L_\varepsilon(f,\vec{z}) := \sup_{\vec{x}\in\mathbb{R}^{d_0}:||\vec{z}-\vec{x}||_2\leq\varepsilon} \{\frac{||f(\vec{z})-f(\vec{x})||_2}{||\vec{z}-\vec{x}||_2}\} \leq l \tag{2.3}$$

Where $L_\varepsilon(f,\vec{z})$ is defined to be the minimal local Lipschitz constant for the function f at the input $\vec{z}$ for the radius of ε. Equivalently, one can say the function f is $(\vec{z},\varepsilon)$-locally l-Lipschitz continuous.

Remark. The Lipschitz constant of a function represents a bound on the rate of change. Namely, if a function is (globally) l-Lipschitz continuous, then perturbations of input $\vec{z}$ of size δ (i.e., $\vec{x} : ||\vec{z}-\vec{x}||_2 \leq \delta$) cannot change the output value by more than $l\delta$. Note that this bound is linear in the distance between the inputs, δ. Therefore, Lipschitz continuity can be thought of as bounding the function by a cone at every point. See figure 2.1 for an illustration of this concept in the unidimensional case. Although the figure illustrates the cone only for one reference point, global Lipschitz continuity implies that it must hold for all points in the domain. Similarly, local Lipschitz continuity requires that it holds for all points in some locality.

2.1.2 Robustness

The definition of global robustness used in this paper is largely inspired by [ABC+24, Definition 4], where the authors formalize the notion of robustness around high confidence regions. In this section, we provide the original definition of robustness alongside our modification.

Definition 2.1.3 (**Confidence-based global 2-safety**). *[ABC+24, Definition 4] A model f is said to be globally 2-safe for confidence $\kappa > 0$ and tolerance $\vec{\varepsilon}$ iff*

$$\forall \vec{x}, \vec{x}'. cond(\vec{x}, \vec{x}', \vec{\varepsilon}) \wedge conf(f(\vec{x})) > \kappa \implies class(f(\vec{x})) = class(f(\vec{x}')) \tag{2.4}$$

Further, the model is confidence-based globally robust if

$$cond(\vec{x}, \vec{x}', \vec{\varepsilon}) = \bigwedge_{i \in [1,n]} (d(x_i, x_i') \leq \varepsilon_i) \tag{2.5}$$

This definition, however, is unable to capture global robustness with respect to the L_2 distance metric as the condition treats the coordinates independently via a logical conjunction. Thus, we adopt a new definition by considering that

$$(\text{class}(f(\vec{x})) = \text{class}(f(\vec{x}'))) \iff (\sigma(f(\vec{x})) \geq 0.5 \iff \sigma(f(\vec{x}')) \geq 0.5) \tag{2.6}$$

Where σ is the sigmoid function. And, defining the confidence in the binary classification case as

$$\text{conf}(f(\vec{x})) = \max\{\sigma(f(\vec{x})), 1 - \sigma(f(\vec{x}))\} \tag{2.7}$$

Definition 2.1.4 (**Global robustness around high-confidence regions**). *A model $f : \mathbb{R}^d \to \mathbb{R}$ is said to be globally ε-robust around κ-confidence regions for a radius of robustness $\varepsilon > 0$ and confidence threshold $\kappa \in (0.5, 1)$ iff*

$$\begin{aligned} &\forall \vec{x}, \vec{x}' \in \mathbb{R}^d, (||\vec{x} - \vec{x}'||_2 \leq \varepsilon \wedge \max\{\sigma(f(\vec{x})), 1 - \sigma(f(\vec{x}))\} \geq \kappa) \\ &\implies (\sigma(f(\vec{x})) \geq 0.5 \iff \sigma(f(\vec{x}')) \geq 0.5) \end{aligned} \tag{2.8}$$

Where σ is the sigmoid function. One can also say that the function f is (ε, κ)-robust.

Remark. The definition of global robustness in high confidence regions formalizes the idea that ε-sized perturbations of a data point that has been classified confidently (confidence above κ) will not change the resulting classification.

2.1.3 Neural Networks

A neural network is a mathematical model consisting of an input layer, an output layer, and L hidden layers ($L \geq 0$). Classically, it is defined as follows. The hidden/output layers are defined as affine transformations that are fed through a non-linear function (called an activation function). Namely, for $l = 1, ..., L+1$ let $g_l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ represent the hidden/output layer l such that for $\vec{x} \in \mathbb{R}^{l-1}$,

$$g_l(\vec{x}) = (\sigma_l \circ A_l)(\vec{x}) \tag{2.9}$$

Where $A_l(\vec{x}) = W_l\vec{x} + \vec{b}_l$ is a linear transformation such that $\vec{b}_l \in \mathbb{R}^{d_l}$ is a bias vector, and $W_l \in \mathbb{R}^{d_{l-1} \times d_l}$ is the weight matrix, both of which are learnable parameters. Moreover, σ_l is a non-linear activation function (such as ReLU, sigmoid, or tanh [GBC16, Section 6.3]) traditionally applied element-wise, but not always (e.g., GroupSort [ALG19]). Each layer transforms its input through this affine mapping followed by a non-linearity, allowing the network to learn complex non-linear functions. Stacking multiple layers enables the neural network to approximate arbitrary functions, provided suitable width and depth, as shown by universal approximation theorems [HSW89].
The entire neural network model, f, can then be represented as a composition of these layer functions:

$$f : \begin{cases} \mathbb{R}^{d_0} & \to \mathbb{R}^{d_{L+1}} \\ \vec{x} & \mapsto (g_{L+1} \circ g_L \circ \cdots \circ g_1)(\vec{x}) \end{cases} \tag{2.10}$$

where $f(\vec{x})$ is the output of the network for input $\vec{x}$, and $L+1$ denotes the total number of layers (including hidden and output layers).

2.1.4 Lipschitz-bounded networks

Lipschitz-bounded networks are a type of neural network with the hypothesis space being a subset of Lipschitz continuous functions (see definition 2.1.1). The Lipschitz constant often is assumed to be 1 for the general definition because global l-Lipschitz continuity can be achieved by multiplying the output by $l \in \mathbb{R}$ ($L(f) = 1 \iff L(lf) = l$). As neural networks can be viewed as a composition of functions, the global Lipschitz bound is typically achieved by bounding individual layers using the following inequality [Don19, Lemma 4],

$$L(f) = L(g_L \circ g_{L-1} \circ \cdots \circ g_1) = L(\sigma_L \circ A_L \circ \sigma_{L-1} \circ \cdots \sigma_1 \circ A_1) \leq (\prod_{l=1}^{L} L(\sigma_l))(\prod_{l=1}^{L} L(A_l)) \tag{2.11}$$

Thus, if for all $l = 1, \ldots, L$ we have that $L(\sigma_l), L(A_l) \leq 1$, then

$$L(f) \leq (\prod_{l=1}^{L} L(\sigma_l))(\prod_{l=1}^{L} L(A_l)) \leq (\prod_{l=1}^{L} 1)(\prod_{l=1}^{L} 1) = 1 \tag{2.12}$$

The proposed method utilizes l-Lipschitz networks to achieve mathematical guarantees. The method is independent of the specific choice of algorithm used to achieve the global Lipschitz bound. As such, we introduce the Spectral Normalization [MKKY18] and the Frobenius Normalization methods due to existing implementations in the *deel-torchlip* Python package [SMGS+20]. In addition, we also introduce the GroupSort and absolute value activation functions in this section due to their gradient-norm-preserving properties, which ensure a tighter Lipschitz constant bound.

Spectral Normalization [MKKY18]. As the Lipschitz constant of a function under the L_2 metric equals the spectral norm of its Jacobian [LRC20], normalizing the function's Jacobian by its spectral norm achieves the exact Lipschitz constant of 1. Namely, we replace weights W_l by their Spectral-normalized counterparts W_l^{SN},

$$W_l^{SN} = \frac{W_l}{||W_l||} \tag{2.13}$$

Where $||W_l||$ is the Spectral norm of a matrix. I.e., let $s_i(W_l)$ denote the i-th singular value of matrix W_l for $i = 1, \ldots, \min\{d_{l-1}, d_l\}$ such that they are sorted in descending order, $i < j \implies s_i(W_l) \geq s_j(W_l)$. Then, the Spectral norm equals the largest singular value,

$$||W_l|| = \sup_{\vec{x}:||\vec{x}||_2=1} ||W_l\vec{x}||_2 = s_1(W_l) \tag{2.14}$$

Thus, for the re-parametrized affine transformation under Spectral Normalization, $A_l^{SN}(\vec{x}) = W_l^{SN}\vec{x} + \vec{b}_l$, we have the following Lipschitz constant,

$$L(A_l^{SN}) = ||W_l^{SN}|| = s_1(W_l^{SN}) = s_1(\frac{W_l}{||W_l||}) = \frac{s_1(W_l)}{||W_l||} = \frac{||W_l||}{||W_l||} = 1 \tag{2.15}$$

Frobenius Normalization. Another approach for ensuring that the Lipschitz constant of a layer is bounded by 1 is to perform Frobenius Normalization. Let $||W_l||_F$ be the Frobenius norm of the matrix W_l,

$$||W_l||_F = \sqrt{\text{trace}(W_l^T W_l)} = \sqrt{\sum_{i=1}^{\min\{d_{l-1}, d_l\}} s_i^2(W_l)} \tag{2.16}$$

It immediately follows that $||W_l|| \leq ||W_l||_F$, hence normalizing the Jacobian by the Frobenius norm will also yield a Lipschitz constant smaller than 1. We define the Frobenius-normalized weights as follows:

$$W_l^{FN} = \frac{W_l}{||W_l||_F} \tag{2.17}$$

Thus, for the re-parametrized affine transformation under Frobenius normalization, $A_l^{FN}(\vec{x}) = W_l^{FN}\vec{x} + \vec{b}_l$, we have the following Lipschitz constant,

$$L(A_l^{FN}) = ||W_l^{FN}|| = s_1(W_l^{FN}) = s_1(\frac{W_l}{||W_l||_F}) = \frac{s_1(W_l)}{||W_l||_F} = \frac{||W_l||}{||W_l||_F} \leq 1 \tag{2.18}$$

Comparison: Spectral and Frobenius Normalization. Spectral normalization provides a direct way of bounding the spectral norm of weight matrices, offering strong theoretical guarantees yet at a higher computational cost. Frobenius normalization, on the other hand, is computationally simpler than Spectral normalization, hence often more practical when computation is a concern.

Activation functions. First, we introduce the concept of norm-preserving functions. A function ϕ is norm-preserving if for all inputs $\vec{x} \in \mathbb{R}^d$ we have that $||\phi(\vec{x})||_2 = ||\vec{x}||_2$. Even though classical activation functions, such as ReLU, sigmoid, and tanh, are 1-Lipschitz, they are not norm-preserving for all inputs. In Lipschitz-bounded layers, controlling the output norm is crucial because it sets limits on how much signals can grow or shrink. Norm-preserving functions preserve the signal strength, which allows the network to make full use of the Lipschitz bound without harming the expressiveness of the layer, therefore are of high importance in the Lipschitz literature. [ALG19]

Such activation functions are GroupSort [ALG19] and the element-wise absolute value function ($x \mapsto |x|$). The GroupSort activation function splits its input into groups of fixed size and sorts each group in ascending order. It is defined as follows:

$$\text{GroupSort} : \mathbb{R}^d \to \mathbb{R}^d, \quad \vec{x} \mapsto \left[\text{sort}(\vec{x}^{(1)}), \text{sort}(\vec{x}^{(2)}), \ldots, \text{sort}(\vec{x}^{(d/k)})\right] \tag{2.19}$$

Where the input vector $\vec{x}$ is split into disjoint groups $\vec{x}^{(i)} \in \mathbb{R}^k$ of fixed size k (with k dividing d), and $\text{sort}(\cdot)$ sorts each group in ascending order independently. GroupSort activation function is shown to be beneficial for Lipschitz-bounded networks. However, it cannot be applied to unidimensional layers. Therefore, in such cases, the absolute value function is used instead.

2.2 Method

2.2.1 Learning task

The proposed method is designed for a binary classification task with multi-dimensional input. Here, we briefly define the learning setup for completeness. Let $\mathcal{X} = (\vec{x}_i \in \mathbb{R}^d)_{i=1}^N$ be our observed features, $\mathcal{Y} = (y_i \in \{0,1\})_{i=1}^N$ the respective labels, and $\mathbb{P}[y|\vec{x}]$ the data generating function. Let $f : \mathbb{R}^d \to \mathbb{R}$ be a model such that $\sigma(f(\vec{x}))$ models $\mathbb{P}[y=1|\vec{x}]$, where $\sigma(a) = \frac{\exp a}{1+\exp a}$ is the sigmoid function.

2.2.2 Proposed method

The proposed method assumes a model parametrized by θ, $f_\theta : \mathbb{R}^d \to \mathbb{R}$, that is decomposable into

$$f_\theta(\vec{x}) = \lambda_\theta(g_\theta(\vec{x}))g_\theta(\vec{x}) \tag{2.20}$$

Where g_θ is globally l_g-Lipschitz, and λ_θ is a positive function. The term g_θ serves as a pre-correction logit (also referred to as initial logit) and captures some notion of model confidence. In addition, g_θ determines the predicted label on its own as λ_θ must always be positive. The term λ_θ maps the aforementioned logits to a new local Lipschitz constant for the function, enabling dynamic local Lipschitz constants based on final confidence ($f_\theta(\vec{x})$). This seemingly *circular* reasoning is possible because the method allows deduction of certain properties of the final output based on the initial logits ($g_\theta(\vec{x})$). This includes knowing whether the final output will be above a confidence threshold.

The method is parametrized by the radius of robustness, $\varepsilon > 0$, and the confidence threshold, $\kappa \in (0.5, 1)$. As the definition of robustness around high confidence thresholds can be modified to only refer to pre-sigmoid activation values (see Corollary 2.3.1.1), κ will often be referred to as the logit-confidence threshold and will take values in $\kappa \in (0, \infty)$ instead. This will be clear from the context.

The proposed method ensures that when the model is confident for a given input, $|f_\theta(\vec{x})| > \kappa$, then the $(\vec{x}, \varepsilon)$-local Lipschitz constant is bounded by $\frac{\kappa}{\varepsilon}$, therefore making it impossible for the model to cross the decision boundary in an ε-radius. This is achieved by the following formulation of the dynamic Lipschitz multiplier network λ_θ:

$$\lambda_\theta(y) = \begin{cases} \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} & : |y| \geq l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \\ \frac{\kappa}{|y|+\delta} n_\theta(y) & : |y| < l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \end{cases} \tag{2.21}$$

Where λ is split into two regions - the confident region ($|y| \geq l_g \varepsilon \frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon$) and the unconfident region. In addition, δ, c are hyperparameters and $n_\theta : \mathbb{R} \to (0,1)$ such that

- $\delta > 0$ controls the maximal rate of change at the decision boundary. This can be seen by considering, $\lambda_\theta(y) \leq \lambda_\theta(0) = \frac{\kappa}{\delta} n_\theta(0) \leq \frac{\kappa}{\delta}$.
- $0 \leq c < \kappa$ where c controls the maximal rate of change around confident regions, and defines the width of the unconfident band.
- n_θ is a learnable model that controls how much of the potential upper bound of the local Lipschitz constant at unconfident regions is used up. I.e., if the optimal $n_\theta(y) \approx 0$ then $\frac{\kappa}{|y|+\delta}$ must be unnecessarily large. This gives the model extra flexibility. This model can be defined as a fully connected neural network, thus, the number of layers and units per layer are also hyperparameters.

This formulation of λ_θ is proved to guarantee global robustness around high confidence regions by Theorem 2.3.9 and Corollary 2.3.9.1.
The proposed method can be applied to any Lipschitz-bounded architecture fit for classification, be it a Convolutional Neural Network, Transformer, etc., as the modifications only occur after the initial logit layer (g). Moreover, the architecture need not be a neural network - the only condition is a known global Lipschitz bound on the original function, g. Thus, this method is model-agnostic.

2.3 Theorems

This section provides the main contributions of the project. Firstly, we show a set of sufficient conditions on any function λ to achieve global robustness around high confidence regions. Then, we show that the specific formulation of λ proposed in section 2.2.2 satisfies these conditions, and, therefore, is globally robust around high confidence regions.

2.3.1 General results

General results will be introduced in this section, whose purpose and application are not limited to the specific method proposed. Namely, this section will cover:

- Preliminary results regarding equivalence of the robustness definition (Proposition 2.3.1, Corollary 2.3.1.1).
- Preliminary results for composition of Lipschitz functions (Propositions 2.3.2, 2.3.3, 2.3.4).

- Sufficient conditions for global robustness around high confidence regions (Lemma 2.3.5).

Proposition 2.3.1. *Monotonicity of the sigmoid. Let* $\sigma(a) = \frac{\exp a}{1+\exp a}$ *be the sigmoid function and* $\kappa \in (0,1)$. *Then*

$$\sigma(a) \geq \kappa \iff a \geq \sigma^{-1}(\kappa) \tag{2.22}$$

Proof. We know that

$$\sigma(a) = \frac{\exp a}{1+\exp a} = \frac{1}{1+\exp(-a)} \geq \kappa \tag{2.23}$$

Hence,

$$\frac{1-\kappa}{\kappa} \geq \exp(-a) \tag{2.24}$$

Now, note that the log function is strictly increasing, thus

$$a \geq \log(\frac{\kappa}{1-\kappa}) = \sigma^{-1}(\kappa) \tag{2.25}$$

□

Remark (Non-technical explanation). This result shows that the sigmoid function is increasing and invertible.

Corollary 2.3.1.1. *Let* $d \in \mathbb{N}$, $f : \mathbb{R}^d \to \mathbb{R}$, $\varepsilon > 0$. *Now, since* $\sigma(0) = 0.5$,

$$\forall \vec{x}, \vec{x}' \in \mathbb{R}^d, (||\vec{x} - \vec{x}'||_2 \leq \varepsilon \wedge |f(\vec{x})| \geq \sigma^{-1}(\kappa)) \implies (f(\vec{x}) \geq 0 \iff f(\vec{x}') \geq 0)$$

$$\iff \tag{2.26}$$

$$\forall \vec{x}, \vec{x}' \in \mathbb{R}^d, (||\vec{x} - \vec{x}'||_2 \leq \varepsilon \wedge \max\{\sigma(f(\vec{x})), 1 - \sigma(f(\vec{x}))\} \geq \kappa)$$
$$\implies (\sigma(f(\vec{x})) \geq 0.5 \iff \sigma(f(\vec{x}')) \geq 0.5)$$

Remark (Non-technical explanation). This corollary shows that we can "push" reasoning about the final confidences back through the sigmoid function, and, hence, can make equivalent deductions only based on the logits. This provides an equivalent form of definition 2.1.4 that is easier to work with.

Proposition 2.3.2. *Local Lipschitz constant of a composition. Let $\lambda, h : \mathbb{R} \to \mathbb{R}$, $g, f : \mathbb{R}^d \to \mathbb{R}$ for some $d \in \mathbb{N}$ be functions such that g is globally Lipschitz continuous, $L(g) = l_g$, $h(y) = \lambda(y)y$, and $f(\vec{x}) = h(g(\vec{x}))$. Assume that for some $\vec{x} \in \mathbb{R}^d$ and $\varepsilon > 0$, h is $(g(\vec{x}), l_g\varepsilon)$-locally Lipschitz continuous, $L_{l_g\varepsilon}(h, g(\vec{x})) < \infty$. Then we have that*

$$L_\varepsilon(f, \vec{x}) \leq l_g L_{l_g\varepsilon}(h, g(\vec{x})) \tag{2.27}$$

Proof. Let $\vec{x}' \in \mathbb{R}^d$ such that $||\vec{x} - \vec{x}'||_2 \leq \varepsilon$. Note that $||\vec{x} - \vec{x}'||_2 \leq \varepsilon \implies ||g(\vec{x}) - g(\vec{x}')|| \leq l_g\varepsilon$, therefore,

$$\frac{||h(g(\vec{x})) - h(g(\vec{x}'))||_2}{||g(\vec{x}) - g(\vec{x}')||_2} \leq L_{l_g\varepsilon}(h, g(\vec{x})) \tag{2.28}$$

Or, equivalently,

$$||h(g(\vec{x})) - h(g(\vec{x}'))||_2 \leq L_{l_g\varepsilon}(h, g(\vec{x}))||g(\vec{x}) - g(\vec{x}')||_2 \tag{2.29}$$

Now, since g is Lipschitz continuous,

$$||g(\vec{x}) - g(\vec{x}')||_2 \leq l_g||\vec{x} - \vec{x}'||_2 \tag{2.30}$$

Thus,

$$||h(g(\vec{x})) - h(g(\vec{x}'))||_2 \leq L_{l_g\varepsilon}(h, g(\vec{x}))l_g||\vec{x} - \vec{x}'||_2 \tag{2.31}$$

Equivalently,

$$\frac{||h(g(\vec{x})) - h(g(\vec{x}'))||_2}{||\vec{x} - \vec{x}'||_2} \leq L_{l_g\varepsilon}(h, g(\vec{x}))l_g \tag{2.32}$$

Now, since this is true for all $\vec{x}'$ such that $||\vec{x}' - \vec{x}||_2 \leq \varepsilon$, we have

$$L_\varepsilon(f, \vec{x}) = \sup_{\vec{x}' : ||\vec{x}' - \vec{x}||_2 \leq \varepsilon} \{\frac{||h(g(\vec{x})) - h(g(\vec{x}'))||_2}{||\vec{x} - \vec{x}'||_2}\} \leq L_{l_g\varepsilon}(h, g(\vec{x}))l_g \tag{2.33}$$

□

Remark (Non-technical explanation). This result shows that the local Lipschitz constant of a composition can be bounded by the product of the individual Lipschitz constants. This helps decompose the Lipschitz constant of the network into simpler terms.

Proposition 2.3.3. *A Lipschitz decomposition. Let $\lambda, h : \mathbb{R} \to \mathbb{R}$ such that $h(y) = \lambda(y)y$. Assume that for some $z \in \mathbb{R}$ and $\delta > 0$, λ is (z, δ)-locally Lipschitz continuous, $L_\delta(\lambda, z) < \infty$.*

Then

$$L_\delta(h,z) \leq |z| L_\delta(\lambda,z) + \max_{y:|y-z|\leq\delta} |\lambda(y)| \tag{2.34}$$

Proof. For any $y \in \mathbb{R}$ we have the following:

$$\begin{aligned} |\lambda(z)z - \lambda(y)y| &= |\lambda(z)z - \lambda(y)z + \lambda(y)z - \lambda(y)y| \\ &\leq |\lambda(z)z - \lambda(y)z| + |\lambda(y)z - \lambda(y)y| \\ &= |z||\lambda(z) - \lambda(y)| + |\lambda(y)||z-y| \\ &= |z||z-y|\frac{|\lambda(z)-\lambda(y)|}{|z-y|} + |\lambda(y)||z-y| \\ &= |z-y|(|z|\frac{|\lambda(z)-\lambda(y)|}{|z-y|} + |\lambda(y)|) \end{aligned} \tag{2.35}$$

Now consider the behaviour more locally. Let $y \in (z-\delta, z+\delta)$. Thus,

$$\begin{aligned} \frac{|\lambda(z)z - \lambda(y)y|}{|z-y|} &\leq |z|\frac{|\lambda(z)-\lambda(y)|}{|z-y|} + |\lambda(y)| \\ &\leq |z| \sup_{y':|y'-z|\leq\delta} \{\frac{|\lambda(z)-\lambda(y')|}{|z-y'|}\} + \max_{y':|y'-z|\leq\delta} |\lambda(y')| \\ &\leq |z| L_\delta(\lambda,z) + \max_{y':|y'-z|\leq\delta} |\lambda(y')| \end{aligned} \tag{2.36}$$

Now, since this holds for all y in a δ-radius around z, we get:

$$L_\delta(h,z) = \sup_{y:|y-z|\leq\delta} \{\frac{|\lambda(z)z - \lambda(y)y|}{|z-y|}\} \leq |z| L_\delta(\lambda,z) + \max_{y:|y-z|\leq\delta} |\lambda(y)| \tag{2.37}$$

□

Remark (Non-technical explanation). This result shows that we can decompose the local Lipschitz constant of the product $\lambda(y)y$ into a sum in a similar style to the derivative product rule. This reduces the problem of bounding the local Lipschitz constant of h to finding properties of the function λ.

Proposition 2.3.4. *Another Lipschitz decomposition. Let $h, \lambda : \mathbb{R} \to \mathbb{R}$ such that $h(y) = y\lambda(y)$, and λ is (y,δ)-locally Lipschitz continuous for some $y \in \mathbb{R}$ and $\delta > 0$ ($L_\delta(\lambda,y) < \infty$). Then,*

$$L_\delta(h,y) \leq L_\delta(\lambda,y)(|y|+\delta) + \lambda(y) \tag{2.38}$$

Proof. By proposition 2.3.3 we know that

$$L_\delta(h,y) \leq |y| L_\delta(\lambda,y) + \max_{y':|y'-y|\leq\delta} |\lambda(y')| \tag{2.39}$$

Now, since λ is (y,δ)-locally Lipschitz continuous, we have that for any $y' : |y-y'| \leq \delta$,

$$\lambda(y') \leq \lambda(y) + \delta L_\delta(\lambda,y) \tag{2.40}$$

Hence,

$$\begin{aligned} L_\delta(h,y) &\leq |y| L_\delta(\lambda,y) + \max_{y':|y'-y|\leq\delta} |\lambda(y')| \\ &\leq |y| L_\delta(\lambda,y) + \lambda(y) + \delta L_\delta(\lambda,y) \\ &= L_\delta(\lambda,y)(|y|+\delta) + \lambda(y) \end{aligned} \tag{2.41}$$

□

Remark (Non-technical explanation). This result builds directly upon Proposition 2.3.3, and simplifies the problem of bounding $L_\delta(h,y)$ further by removing the max operator by using Lipschitz properties instead.

Lemma 2.3.5. *Sufficient conditions of the dynamic Lipschitz constant for global robustness around high confidence regions. Let $\kappa > 0$ be the logit-confidence threshold. Let $\lambda, b : \mathbb{R} \to \mathbb{R}$ be functions such that for some $\delta > 0$ we have the following:*

1. *Some positive value on the domain of $\lambda(y)y$ attains the logit-confidence threshold. In addition, this input is large enough to allow inputs in a δ-radius not to cross the decision boundary ($h(0) = \lambda(0)0 = 0$). $\exists y_0 > \delta : \lambda(y_0)y_0 = \kappa$.*

2. *λ is a symmetric function. $\forall y, \lambda(y) = \lambda(-y)$.*

3. *$\lambda(y)y$ is increasing in y. $\forall y' > y > \delta \implies \lambda(y')y' - \lambda(y)y > 0$.*

4. *The (y',δ)-local Lipschitz constant is bounded for all inputs y' above a certain threshold. Let $h_\lambda(y) = \lambda(y)y$ then $\forall y' : y' > y_0 - \delta$ we have $L_\delta(h_\lambda, y') \leq \frac{\kappa}{\delta}$.*

5. *Function values on the domain near the decision boundary are not confident. $|y| < y_0 - \delta \implies |b(y)y| < \kappa$.*

Let $h, s : \mathbb{R} \to \mathbb{R}$ be functions, such that $h(y) = s(y)y$ and

$$s(y) = \begin{cases} \lambda(y) & : |y| \geq y_0 - \delta \\ b(y) & : |y| < y_0 - \delta \end{cases} \tag{2.42}$$

Then for $f, g : \mathbb{R}^d \to \mathbb{R}$ such that $f(\vec{x}) = h(g(\vec{x}))$ where $g(\vec{x})$ is l_g-Lipschitz, we have that for $\varepsilon = \frac{\delta}{l_g}$, f is ε-robust around $\sigma(\kappa)$-confidence regions.

Proof. The proof is constructed as follows. Only non-negative values of $g(\vec{z}) = y$ are considered because the negative case is derived from symmetry. First, it is shown that $0 \leq y < y_0$ results in unconfident predictions ($0 \leq h(y) < \kappa$). Therefore, the logical statement does not apply. Second, it is shown that the result applies for values $y \geq y_0$.
Let $y = g(\vec{z})$ and $y' = g(\vec{x})$ for some $\vec{x} : |\vec{x} - \vec{z}| \leq \varepsilon$. First, assume that $0 \leq y < y_0 - \delta$. Then due to assumption 5, $f(\vec{z}) = h(y) \in (-\kappa, \kappa)$, hence the statement does not apply.
Now consider, $y_0 - \delta \leq y < y_0$. Due to assumption 4, we know that h is locally Lipschitz. In addition, due to assumption 1, we have $h(y_0) = \kappa$. Therefore, $h(y_0 - \delta) > h(y_0) - \delta L_\delta(h_\lambda, y_0) \geq \kappa - \delta\frac{\kappa}{\delta} = 0$. Moreover, due to assumption 3, h_λ is strictly increasing, thus,

$$0 \leq h(y_0 - \delta) \leq h(y) < h(y_0) = \kappa \tag{2.43}$$

Therefore, $f(\vec{z}) = h(y) \in [0, \kappa)$, hence the statement does not apply.
Now consider $g(\vec{z}) = y \geq y_0$. Due to assumption 3, $h(y) \geq h(y_0) = \kappa$, thus the statement applies. Now, consider $y' = g(x) : |\vec{x} - \vec{z}| \leq \varepsilon$. Since g is l_g-Lipschitz, $|y' - y| \leq l_g\varepsilon = l_g\frac{\delta}{l_g} = \delta$. Now, we know that $y_0 - \delta \leq y - \delta \leq y'$. Thus,

$$0 \leq h(y_0 - \delta) \leq h(y') = f(x) \tag{2.44}$$

By symmetry (assumption 2), we have the same result for $g(\vec{z}) = y < 0$. Hence,

$$f(\vec{z}) > \kappa \implies \forall \vec{x} : |\vec{x} - \vec{z}| \leq \varepsilon, f(\vec{x}) \geq 0 \tag{2.45}$$

$$f(\vec{z}) < -\kappa \implies \forall \vec{x} : |\vec{x} - \vec{z}| \leq \varepsilon, f(\vec{x}) \leq 0 \tag{2.46}$$

Thus, by Corollary 2.3.1.1 we have global ε-robustness around regions of at least $\sigma(\kappa)$ confidence,

$$\begin{aligned} &\forall \vec{x}, \vec{x}' \in \mathbb{R}^d, (||\vec{x} - \vec{x}'||_2 \leq \varepsilon \wedge \max\{\sigma(f(\vec{x})), 1 - \sigma(f(\vec{x}))\} \geq \kappa) \\ &\implies (\sigma(f(\vec{x})) \geq 0.5 \iff \sigma(f(\vec{x}')) \geq 0.5) \end{aligned} \tag{2.47}$$

□

Remark (Non-technical explanation). This result shows a way to decompose the function f over its domain into two functions. A set of conditions for these functions is defined, which are shown to be sufficient for global robustness around high confidence regions.

2.3.2 Method

After having introduced the general results in section 2.3.1, we move on to the main result of this dissertation - the proof of robustness of the proposed method.
The proposed λ that is proven to achieve guaranteed robustness in this section has the following functional form:

$$\lambda(y) = \begin{cases} \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} & : |y| \geq l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \\ \frac{\kappa}{|y|+\delta}n(y) & : |y| < l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \end{cases}$$

Such that $\kappa > 0$ is the logit-confidence threshold, $\varepsilon > 0$ is the radius of robustness, $l_g > 0$ is the global Lipschitz constant of the initial logit network (g in $f(x) = g(x)\lambda(g(x))$), and $c, \delta > 0$ are free parameters such that $\kappa - c > 0$.

Proposition 2.3.6. *Let $\kappa, d, \delta, c \in \mathbb{R}$ and $\varepsilon > 0$ such that $d + \delta > 0$. Let $\lambda(y) = \frac{\kappa}{\delta} + \frac{c-\kappa}{|y|+d+\delta}$. Then for $y \in \mathbb{R}$ such that $|y| > \varepsilon$,*

$$L_\varepsilon(\lambda, y) \leq \frac{|c-\kappa|}{(|y|+d+\delta)(|y|-\varepsilon+d+\delta)} \tag{2.48}$$

Proof. For any $y' \in \mathbb{R}$ we have that

$$\begin{aligned} \frac{|\lambda(y)-\lambda(y')|}{|y-y'|} &= \frac{|\frac{\kappa}{\delta} + \frac{c-\kappa}{|y|+d+\delta} - \frac{\kappa}{\delta} - \frac{c-\kappa}{|y'|+d+\delta}|}{|y-y'|} \\ &= \frac{|\frac{c-\kappa}{|y|+d+\delta} - \frac{c-\kappa}{|y'|+d+\delta}|}{|y-y'|} \\ &= |c-\kappa|\frac{|\frac{1}{|y|+d+\delta} - \frac{1}{|y'|+d+\delta}|}{|y-y'|} \\ &= |c-\kappa|\frac{|\frac{|y'|+d+\delta-(|y|+d+\delta)}{(|y|+d+\delta)(|y'|+d+\delta)}|}{|y-y'|} \\ &= |c-\kappa|\frac{\frac{||y'|-|y||}{|(|y|+d+\delta)(|y'|+d+\delta)|}}{|y-y'|} \\ &= \frac{|c-\kappa|}{|(|y|+d+\delta)(|y'|+d+\delta)|}\frac{||y'|-|y||}{|y-y'|} \end{aligned} \tag{2.49}$$

Now, assume $|y'-y| \leq \varepsilon$. Combining this with $|y| - \varepsilon > 0$, we have that $\text{sign}(y) = \text{sign}(y')$,

thus $||y|-|y'|| = |\text{sign}(y)y - \text{sign}(y')y'| = |\text{sign}(y)(y-y')| = |y-y'|$. Hence,

$$\begin{aligned}\frac{|\lambda(y)-\lambda(y')|}{|y-y'|} &= \frac{|c-\kappa|}{|(|y|+d+\delta)(|y'|+d+\delta)|}\frac{||y'|-|y||}{|y-y'|} \\ &= \frac{|c-\kappa|}{|(|y|+d+\delta)(|y'|+d+\delta)|}\frac{|y-y'|}{|y-y'|} \\ &= \frac{|c-\kappa|}{||y|+d+\delta|\,||y'|+d+\delta|}\end{aligned} \tag{2.50}$$

Note that,

$$(|y'-y| \leq \varepsilon) \wedge (|y|-\varepsilon > 0) \implies 0 < |y|-\varepsilon \leq |y'| \tag{2.51}$$

Therefore,

$$0 < d+\delta < |y|-\varepsilon+d+\delta \leq |y'|+d+\delta \tag{2.52}$$

Thus,

$$\frac{1}{||y'|+d+\delta|} = \frac{1}{|y'|+d+\delta} \leq \frac{1}{|y|-\varepsilon+d+\delta} = \frac{1}{||y|-\varepsilon+d+\delta|} \tag{2.53}$$

Hence, we can make the rate of change bound independent of y',

$$\frac{|\lambda(y)-\lambda(y')|}{|y-y'|} = \frac{|c-\kappa|}{||y|+d+\delta|\,||y'|+d+\delta|} \leq \frac{|c-\kappa|}{(|y|+d+\delta)(|y|-\varepsilon+d+\delta)} \tag{2.54}$$

Since this holds for all y' in an ε-radius around y,

$$L_\varepsilon(\lambda, y) \leq \frac{|c-\kappa|}{(|y|+d+\delta)(|y|-\varepsilon+d+\delta)} \tag{2.55}$$

□

Remark (Non-technical explanation). This proposition shows an analytical solution to the local Lipschitz constant of the specific functional form of $\lambda(y)$. This enables using previous results, e.g., Proposition 2.3.4, to bound the Lipschitz constant of $f(x) = \lambda(g(x))g(x)$.

Proposition 2.3.7. *Let $f, g : \mathbb{R}^d \to \mathbb{R}$, $h, \lambda : \mathbb{R} \to \mathbb{R}$ such that $h(y) = y\lambda(y)$ and $f(\vec{x}) = h(g(\vec{x}))$, and g is globally l_g-Lipschitz. Let $\kappa, \varepsilon > 0$, and $c \in \mathbb{R}$ such that $0 < c < \kappa$. Then for*

$$\lambda(y) = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} \tag{2.56}$$

We have that for all $\vec{x} \in \mathbb{R}^d$,

$$L_\varepsilon(f,\vec{x}) \leq \frac{\kappa}{\varepsilon} \tag{2.57}$$

Proof. Since f is a composition of functions, we apply Proposition 2.3.2,

$$L_\varepsilon(f,\vec{x}) \leq l_g L_{l_g\varepsilon}(h, g(\vec{x})) \tag{2.58}$$

Now, by Proposition 2.3.4,

$$L_{l_g\varepsilon}(h, g(\vec{x})) \leq L_{l_g\varepsilon}(\lambda, g(\vec{x}))(|g(\vec{x})| + \varepsilon l_g) + \lambda(g(\vec{x})) \tag{2.59}$$

By Proposition 2.3.6 we have the following Lipschitz bound for λ:

$$L_{l_g\varepsilon}(\lambda, g(\vec{x})) \leq \frac{|c-\kappa|}{(|y| + l_g\varepsilon + l_g\varepsilon)(|y| - l_g\varepsilon + l_g\varepsilon + l_g\varepsilon)} = \frac{|c-\kappa|}{(|y| + 2l_g\varepsilon)(|y| + l_g\varepsilon)} \tag{2.60}$$

Hence,

$$\begin{aligned} L_{\varepsilon l_g}(\lambda, g(\vec{x}))(|g(\vec{x})| + \varepsilon l_g) + \lambda(g(\vec{x})) \leq{}& \frac{|c-\kappa|}{(|g(\vec{x})| + 2l_g\varepsilon)(|g(\vec{x})| + l_g\varepsilon)}(|g(\vec{x})| + \varepsilon l_g) \\ &+ \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|g(\vec{x})| + 2l_g\varepsilon} \end{aligned} \tag{2.61}$$

Now, since $c < \kappa$, we have that $c - \kappa = -|c-\kappa|$

$$\begin{aligned} L_{\varepsilon l_g}(\lambda, g(\vec{x}))(|g(\vec{x})| + \varepsilon l_g) + \lambda(g(\vec{x})) \leq{}& \frac{|c-\kappa|}{|g(\vec{x})| + 2l_g\varepsilon} + \frac{\kappa}{l_g\varepsilon} - \frac{|c-\kappa|}{|g(\vec{x})| + 2l_g\varepsilon} \\ ={}& \frac{\kappa}{l_g\varepsilon} \end{aligned} \tag{2.62}$$

Thus,

$$L_\varepsilon(f,\vec{x}) \leq l_g L_{l_g\varepsilon}(h, g(\vec{x})) \leq l_g \frac{\kappa}{l_g\varepsilon} = \frac{\kappa}{\varepsilon} \tag{2.63}$$

□

Remark (Non-technical explanation). This crucial result shows the ε-local Lipschitz constant of the neural network, f. This is a necessary step for Lemma 2.3.5 (assumption 4).

Proposition 2.3.8. *Let* $\kappa, c, l_g, \varepsilon \in \mathbb{R}$ *such that* $0 \leq c < \kappa$. *Let* $y, y' \in \mathbb{R}$. *Then for*

$$\lambda(y) = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y| + 2l_g\varepsilon} \tag{2.64}$$

We have that $h(y) = y\lambda(y)$ is increasing as follows,

$$(0 \leq y < y' \implies h(y) < h(y')) \wedge (y < y' \leq 0 \implies h(y) < h(y')) \tag{2.65}$$

Proof.

$$\begin{aligned} h(y') - h(y) &= y'\lambda(y') - y\lambda(y) \\ &= \frac{\kappa}{l_g\varepsilon}y' + \frac{c-\kappa}{|y'|+2l_g\varepsilon}y' - \frac{\kappa}{l_g\varepsilon}y - \frac{c-\kappa}{|y|+2l_g\varepsilon}y \\ &= \frac{\kappa}{l_g\varepsilon}(y'-y) + (c-\kappa)(\frac{y'}{|y'|+2l_g\varepsilon} - \frac{y}{|y|+2l_g\varepsilon}) \\ &= \frac{\kappa}{l_g\varepsilon}(y'-y) + (c-\kappa)(\frac{|y|y'+2l_g\varepsilon y' - |y'|y - 2l_g\varepsilon y}{(|y|+2l_g\varepsilon)(|y'|+2l_g\varepsilon)}) \end{aligned} \tag{2.66}$$

Now, note that $\text{sign}(y) = \text{sign}(y') \implies |y|y' = |y'|y$. Thus,

$$\begin{aligned} y'\lambda(y') - y\lambda(y) &= \frac{\kappa}{l_g\varepsilon}(y'-y) + 2l_g\varepsilon(c-\kappa)(\frac{y'-y}{(|y|+2l_g\varepsilon)(|y'|+2l_g\varepsilon)}) \\ &= (y'-y)[\frac{\kappa}{l_g\varepsilon} + (c-\kappa)\frac{2l_g\varepsilon}{(|y|+2l_g\varepsilon)(|y'|+2l_g\varepsilon)}] \end{aligned} \tag{2.67}$$

Now, consider the terms. Note that since $y' > y$, we have that $\text{sign}(y'-y) = +1$. Now, we want to determine the sign of the other term. Note that $0 \leq c < \kappa \implies (c-\kappa) \leq 0$. Hence, we have that:

$$|y'| + 2l_g\varepsilon > 2l_g\varepsilon > 0 \implies \frac{1}{2l_g\varepsilon} > \frac{1}{|y'|+2l_g\varepsilon} \implies \frac{c-\kappa}{|y'|+2l_g\varepsilon} > \frac{c-\kappa}{2l_g\varepsilon} \tag{2.68}$$

Thus,

$$\begin{aligned} \frac{\kappa}{l_g\varepsilon} + (c-\kappa)\frac{2l_g\varepsilon}{(|y|+2l_g\varepsilon)(|y'|+2l_g\varepsilon)} &\geq \frac{\kappa}{l_g\varepsilon} + (c-\kappa)\frac{2l_g\varepsilon}{(2l_g\varepsilon)^2} \\ &= \frac{2\kappa}{2l_g\varepsilon} + \frac{c-\kappa}{2l_g\varepsilon} \\ &= \frac{\kappa+c}{2l_g\varepsilon} \\ &> 0 \end{aligned} \tag{2.69}$$

Therefore, this term does not affect the sign. Thus,

$$\text{sign}(y'\lambda(y') - y\lambda(y)) = \text{sign}(y'-y) = +1 \tag{2.70}$$

Therefore,

$$(0 \leq y < y' \implies h(y) < h(y')) \wedge (y < y' \leq 0 \implies h(y) < h(y')) \tag{2.71}$$

□

Remark (Non-technical explanation). This result shows that the proposed network function is increasing with initial logits, and is a necessary step for Lemma 2.3.5 (assumption 3).

Theorem 2.3.9. *Let* $c, \kappa, l_g, \varepsilon > 0$ *such that* $0 < c < \kappa$. *Let* $g : \mathbb{R}^d \to \mathbb{R}$ *be* l_g*-Lipschitz. Let* $\lambda(y)$ *be defined as follows:*

$$\lambda(y) = \begin{cases} \frac{\kappa}{l_g \varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} & : |y| \geq l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \\ b(y) & : |y| < l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \end{cases} \tag{2.72}$$

Where $b(y)$ *is any positive function such that for* $y \in (-l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - 1), l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - 1))$, $|b(y)y| < \kappa$. *Then* $f(\vec{x}) = \lambda(g(\vec{x}))g(\vec{x})$ *guarantees* ε*-robustness around regions with confidence of at least* $\sigma(\kappa)$*:*

$$\forall \vec{x}, \vec{x}' \in \mathbb{R}^d, (||\vec{x} - \vec{x}'||_2 \leq \varepsilon \wedge \max\{\sigma(f(\vec{x})), 1 - \sigma(f(\vec{x}))\} \geq \kappa) \tag{2.73}$$

$$\implies (\sigma(f(\vec{x})) \geq 0.5 \iff \sigma(f(\vec{x}')) \geq 0.5) \tag{2.74}$$

Proof. Let $h(y) = \lambda(y)y$ such that $f(\vec{x}) = h(g(\vec{x}))$. Let $y_0 = l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa}$ and $\delta = l_g\varepsilon$. Let $D_\lambda = \{y : |y| \geq y_0 - \delta\}$, i.e., the sub-domain on which λ takes the function form $\lambda(y) = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon}$. In addition, let $D_b = \mathbb{R} \setminus D_\lambda$. Then the defined function satisfies all the requirements of Lemma 2.3.5. Namely,

1) The function $\lambda(y)y$ attains the confidence threshold at $y = y_0$:

$$\begin{aligned}
\lambda(y_0)y_0 &= \frac{\kappa}{l_g\varepsilon}y_0 + \frac{c-\kappa}{|y_0|+2l_g\varepsilon}y_0 \\
&= \frac{\kappa}{l_g\varepsilon}y_0 + \frac{c-\kappa}{y_0+2l_g\varepsilon}y_0 \\
&= \frac{\kappa y_0(y_0+2l_g\varepsilon)+(c-\kappa)y_0 l_g\varepsilon}{l_g\varepsilon(y_0+2l_g\varepsilon)} \\
&= \frac{\kappa y_0^2+2l_g\varepsilon\kappa y_0+cy_0l_g\varepsilon-\kappa y_0 l_g\varepsilon}{l_g\varepsilon(y_0+2l_g\varepsilon)} \\
&= \frac{\kappa y_0^2+y_0 l_g\varepsilon(c+\kappa)}{l_g\varepsilon(y_0+2l_g\varepsilon)} \\
&= \frac{\kappa(l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa})^2+(l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa})l_g\varepsilon(c+\kappa)}{l_g\varepsilon(l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa}+2l_g\varepsilon)} \\
&= \frac{\kappa\frac{1}{4\kappa^2}(l_g\varepsilon)^2(\sqrt{c^2+8\kappa^2}-c)^2+(l_g\varepsilon)^2\frac{1}{2\kappa}(\sqrt{c^2+8\kappa^2}-c)(c+\kappa)}{(l_g\varepsilon)^2(\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa}+2)} \\
&= \frac{(\sqrt{c^2+8\kappa^2}-c)^2+2(\sqrt{c^2+8\kappa^2}-c)(c+\kappa)}{4\kappa(\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa}+2)} \\
&= \frac{c^2+8\kappa^2+c^2-2c\sqrt{c^2+8\kappa^2}+2c\sqrt{c^2+8\kappa^2}+2\kappa\sqrt{c^2+8\kappa^2}-2c^2-2c\kappa}{2(\sqrt{c^2+8\kappa^2}-c)+8\kappa} \\
&= \frac{8\kappa^2+2\kappa\sqrt{c^2+8\kappa^2}-2c\kappa}{2(\sqrt{c^2+8\kappa^2}-c)+8\kappa} \\
&= \kappa
\end{aligned} \tag{2.75}$$

In addition, $y_0 > l_g\varepsilon$:

$$\begin{aligned}
y_0 - l_g\varepsilon &= l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa}-1) \\
&= l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-c-2\kappa}{2\kappa}) \\
&= l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-\sqrt{(c+2\kappa)^2}}{2\kappa}) \\
&= l_g\varepsilon(\frac{\sqrt{c^2+8\kappa^2}-\sqrt{c^2+4\kappa^2+4c\kappa}}{2\kappa})
\end{aligned} \tag{2.76}$$

Now, since $c < \kappa$, we have that $4\kappa^2 > 4c\kappa$. Thus, $c^2+8\kappa^2 > c^2+4\kappa^2+4c\kappa$. Hence,

$$y_0 - l_g\varepsilon \geq 0 \tag{2.77}$$

2) The function λ is symmetric on the subdomain D_λ:

$$\lambda(y) = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|-y|+2l_g\varepsilon} = \lambda(-y) \tag{2.78}$$

3) The function $\lambda(y)y$ is increasing in y on the subdomain D_λ by Proposition 2.3.8.
4) The function $h(y) = \lambda(y)y$ has a local Lipschitz constant for all y': $L_{l_g\varepsilon}(h,y') \leq \frac{\kappa}{\varepsilon l_g}$ by Proposition 2.3.7.
5) The function $b(y)y$ on the subdomain D_b is bounded by the confidence threshold. This is true by assumption, i.e., $y \in D_b \implies |b(y)y| < \kappa$
Thus, by Lemma 2.3.5, $f(g(x)) = \lambda(g(x))g(x)$ is ε-robust in $\sigma(\kappa)$-confidence regions. □

Corollary 2.3.9.1. *A choice of $b(y)$.*
Let

$$\lambda(y) = \begin{cases} \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon} & : |y| \geq l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \\ b(y) & : |y| < l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon \end{cases} \tag{2.79}$$

Where for some hyperparameter $\delta > 0$ and some function $n : \mathbb{R} \to [0,1]$,

$$b(y) = \frac{\kappa}{|y|+\delta} n(y) \tag{2.80}$$

Then this λ satisfies all requirements as $|b(y)y| = \frac{\kappa}{|y|+\delta} n(y)|y| \leq \kappa n(y) \leq \kappa$. As long as the codomain of n is a subset of $[0,1]$, n can be any function, e.g., a neural network model.

Remark (Non-technical explanation). Here we have applied the more general Lemma 2.3.5 to our method to show robustness around high confidence regions. This final result completes our proof.

Chapter 3

Results and Evaluation

3.1 Experimental setup

This section describes the experimental setup, where the following research hypotheses and questions are used as guidance for the experiments:

1. **Hypothesis**: the proposed method achieves higher accuracy than Lipschitz-bounded networks while being more robust due to having dynamic local Lipschitz constants.

2. **Research question**: how to ensure that the model does not choose trivial (non-confident) solutions?

3. **Research question**: how do hyperparameters affect the fitted network?

All models in the study are trained using the stochastic gradient descent (SGD) algorithm [Rud17] without using momentum or weight decay for simplicity. For all models, SGD is employed to minimize the binary cross-entropy training objective. In addition, all experiments are conducted with the radius of robustness $\varepsilon = 8/255$ as is common in the literature, and the probability-based confidence threshold $\sigma(\kappa) = 0.8$.

3.1.1 Benchmark Data

To empirically evaluate the proposed method, a synthetic unidimensional dataset generated from a normalized sigmoid function with slope 20 is employed. Using this dataset allows for focused testing of the theoretical properties of the model. Moreover, it allows visualization due to the unidimensional domain and codomain.
The sigmoid is normalized such that the inflection point occurs at $x = 0.5$, and the domain is limited to $[0, 1]$. This is done to imitate the common perturbation setting in the adversarial attack literature. Namely, many methods often deal with image data, where pixels take values

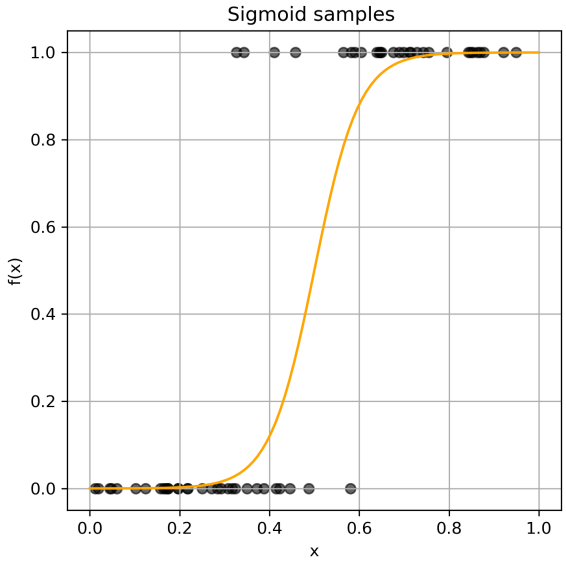


Figure 3.1: Plot of $f(x) = \sigma(20(x-0.5))$. Additionally, 60 points are sampled from the function and plotted as black dots.

in $[0,1]$. The normalized sigmoid function is formulated as follows:

$$f(x) = \sigma(s(x-0.5)) = \frac{1}{1+\exp(-s(x-0.5))} \tag{3.1}$$

Where s is the slope and σ is the sigmoid function. The slope is chosen to be $s = 20$ as this gives a clear separation boundary between classes, while having enough aleatoric uncertainty near $x = 0.5$ to make the model fitting problem non-trivial. See figure 3.1 for an illustration of the data-generating function. Datasets are generated using the following algorithm:

1. Randomly sample the uniform distribution from the domain (0 to 1), $z_i \sim \text{Uniform}(0,1)$
2. Map these samples to probabilities via the sigmoid function, $p_i = \sigma(s(z_i - 0.5))$
3. Generate the final data by sampling from the corresponding Bernoulli distribution, $x_i \sim \text{Bernoulli}(p_i)$

For each run 200 train, 100 test, and 100 validation points are sampled independently.

3.1.2 Baseline Models

In order to benchmark the proposed method against a representative set of existing approaches, three classes of models are selected for comparison: fully connected neural networks, Lipschitz-constrained neural networks, and adversarially trained models.

Fully connected neural networks (FCNNs). Standard feedforward networks with ReLU activations serve as a natural baseline because they represent the unconstrained architecture

with no robustness incentives or guarantees. Including them allows us to measure the effect of robustness-oriented modifications.

Lipschitz-bounded networks. Lipschitz-bounded networks use a fixed global Lipschitz constant, which guarantees robustness but often at the cost of flexibility. Since this project focuses on replacing the global bound with a dynamic local Lipschitz constant, including standard Lipschitz models provides a direct way to measure the trade-offs. For both Lipschitz-bounded networks and our proposed method, Frobenius normalization is used to ensure a Lipschitz bound because this proved to be more numerically stable and efficient than Spectral normalization. In addition, GroupSort/absolute value activation functions are used for both models due to reasons discussed in Section 2.1.4.

Adversarial training. Adversarial training, unlike Lipschitz models or the proposed method, does not constrain the network, nor does it have any robustness guarantees. It improves robustness purely through exposure to attacks. This makes it a good comparison of how the proposed method compares to an unconstrained empirical method. It is set up exactly the same as FCNNs. However, the loss function is modified using the FGSM attack with $\alpha = 0.5$ (see section 1.2.1). FGSM attack was chosen as it is a widely studied baseline attack, simple to implement, and commonly used as a first benchmark for evaluating robustness in the literature.

3.1.3 Evaluation Metrics

A model that is robust in high-confidence areas is not necessarily useful. If all its predictions fall in low-confidence areas, then the robustness results are meaningless. To take full advantage of the robustness theory, the model should maximize the data points it assigns high confidence to while preserving accuracy.

The loss function (binary cross-entropy) already incentivises high confidence scores for regions where one class dominates. However, that may not be enough to ensure that the proposed model ever enters high confidence regions, as it does not take into consideration the confidence threshold parameter, κ. Therefore, empirical coverage is proposed to be used as an additional evaluation metric. Let f_θ be the model, κ the logit-confidence threshold, and $\mathcal{D} = ((\vec{x}_i, y_i))_{i=1}^n$ a dataset of features ($\vec{x}_i$) and labels (y_i). Then the coverage is defined as follows:

$$\begin{aligned} \text{coverage}(f_\theta, \kappa, \mathcal{D}) &= \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}(\text{confidence}(f_\theta, \vec{x}_i) > \sigma(\kappa)) \\ &= \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}(|f_\theta(\vec{x}_i)| \geq \kappa) \end{aligned} \tag{3.2}$$

Where $\mathbb{I}$ is an indicator function (1 if the boolean expression in its argument is true, and 0 if false). Coverage captures the empirical probability that the model f_θ is κ-confident given a

random point from the dataset $\mathcal{D}$.
In addition, we also measure the average confidence on the test set, which complements the coverage metric and informs us of how confident the predictions are. It is defined as follows:

$$\text{confidence}(f_\theta, \mathcal{D}) = \frac{1}{n}\sum_{i=1}^{n} \text{confidence}(f_\theta, \vec{x}_i) \tag{3.3}$$

Thirdly, we measure the empirical robustness at some perturbation radius ε under the FGSM attack. It is defined as follows:

$$\text{ER}(f_\theta, \mathcal{D}, \varepsilon) = \frac{\sum_{i=1}^{n} \mathbb{I}(|f_\theta(\vec{x}_i)| > \kappa \wedge [\text{class}(f_\theta(\vec{x}_i)) = \text{class}(f_\theta(FGSM(\vec{x}_i; f_\theta, \varepsilon)))])}{\sum_{i=1}^{n} \mathbb{I}(|f_\theta(\vec{x}_i)| > \kappa)} \tag{3.4}$$

Where $FGSM(\vec{x}_i; f_\theta, \varepsilon)$ is the adversarial example acquired by applying the FGSM attack at the input $\vec{x}_i$ and perturbation radius ε. Empirical robustness measures the proportion of confident points, which do not change classification under an FGSM-based ε-perturbation. This allows us to (1) verify that the method is working, and (2) test whether the method transfers the robustness to larger perturbation radii, which it was not trained for.
Moreover, we use the accuracy metric to determine the predictive quality of our model. It is defined as follows:

$$\text{Accuracy}(f_\theta, \mathcal{D}) = \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}(\text{class}(f_\theta(\vec{x}_i)) = y_i) \tag{3.5}$$

In addition to the proposed metrics, we also utilize the negative log-likelihood of the Bernoulli distribution, namely the binary cross-entropy loss function as our main model selection and evaluation metric. It is formulated as follows:

$$\text{BCE}(f_\theta, \mathcal{D}) = -\frac{1}{n}\sum_{i=1}^{n} [y_i \log(f_\theta(\vec{x}_i)) + (1 - y_i)\log(1 - f_\theta(\vec{x}_i))] \tag{3.6}$$

3.2 Experiments

Having defined the datasets, baseline models, hypothesis, and evaluation metrics, we now move on to the two experiments. The first experiment will aim to answer hypothesis 1 and research question 2, while the second experiment will answer research question 3.
All experiments were conducted on an Apple M2 MacBook Pro running macOS Sequoia 15.6 with 16GB RAM, using Python 3.10.18, PyTorch 2.7.1, and deel-torchlip 1.0.1.

Table 3.1: Grid of hyperparameters explored in model selection. The grid is applied to (1) the proposed method, (2) FCNN, (3) adversarially trained networks, and (4) Lipschitz-bounded networks.

Hyperparameter	**Range**	**Applicable models**
Hidden size	$\{1,2,4\}$	All models
Number of hidden layers	$\{1,2\}$	All models
Learning rate (SGD)	$\{10^{-3},10^{-2},10^{-1},5\cdot10^{-1},1\}$	All models
Batch size	$\{16\}$	All models
Lipschitz constant	$\{10^{-1},10^{0},10^{1},10^{2}\}$	Lipschitz / Proposed method
δ	$\{10^{-2},10^{-1},10^{0}\}$	Proposed method
c	$\{0,\frac{1}{10}\sigma^{-1}(\kappa),\frac{1}{2}\sigma^{-1}(\kappa)\}$	Proposed method
Hidden size (n_θ)	$\{1,2,4\}$	Proposed method
Number of hidden layers (n_θ)	$\{1,2\}$	Proposed method

3.2.1 Experiment 1

Experimental setup. The first set of experiments is carried out on the synthetic dataset, enabling controlled testing of robustness and approximation while retaining interpretability. The dynamical Lipschitz network is compared against the three baseline model families (FCNNs, Lipschitz networks, adversarially trained models).

Training setup. All models are trained using stochastic gradient descent (SGD) over a limited grid of hyperparameters. The particular choice of the grid is based on the simplicity of the data-generating function and the necessity to cover a wide range of hyperparameters for the proposed method. The full grid is summarized in Table 3.1. Due to computational limitations, batch size is not tuned, but is chosen to be 16 as is common in literature. Similarly, momentum and weight decay are not used.

For each hyperparameter configuration, models are trained three independent times for 30 epochs. The mean binary cross entropy loss and the standard deviation are calculated on a validation set consisting of 100 points, independently sampled from the same distribution. The model hyperparameters with the lowest mean loss are selected as the best set of hyperparameters. Further, the best model is trained 30 independent times for 100 epochs and evaluated on independent test sets. The evaluation metrics are aggregated across the runs to limit stochasticity.

Results. Using the mean loss of the three runs, the best performing hyperparameters were the following:

- FCNN: hidden size 4, number of hidden layers 1, learning rate 0.5
- Adversarial Training: hidden size 4, number of hidden layers 1, learning rate 0.5
- Lipschitz-bounded network: Lipschitz constant 10, hidden size 2, number of hidden

layers 2, learning rate 10^{-3}

- Proposed method: l_g parameter 1, c parameter $0.5\sigma^{-1}(\kappa)$, δ parameter 10^{-2}, hidden size 2, number of hidden layers 1, unconfident network (n_θ) number of layers 1, unconfident network (n_θ) hidden size 4, learning rate 10^{-3}

From 30 runs, some outliers were observed in both adversarial and FCNN models, with performance close to random chance ($\approx 50\%$ accuracy). These outliers (3 for FCNN and 5 for adversarial) were excluded from subsequent analysis as we hypothesize that this may be attributed to the ReLU activation, which requires careful weight initialization (such as He or Xavier methods) to avoid poor training outcomes [GB10, HZRS15]. In contrast, no outliers appeared in the Lipschitz/proposed model using GroupSort and absolute value activations, possibly due to the norm-preserving properties of these activations. Table 3.2 summarizes the evaluation metrics after applying this model selection approach. See Figure 3.2 for an example of a fitted model under the best-performing hyperparameters.

Although the proposed model achieves the highest mean accuracy (94.6%) with the lowest mean loss (0.149), these results are not conclusive as these metrics are within one standard deviation of values from other models. Thus, we have only shown partial support for hypothesis 1, without conclusive evidence.

Most models were empirically robust for perturbations of size $1/255$ and $8/255$, but the proposed method showed improvements in robustness, even reaching far outside of the radius of robustness (91.8% robust for perturbations of size $32/255$). However, again, the results are not conclusive due to the large standard deviations.

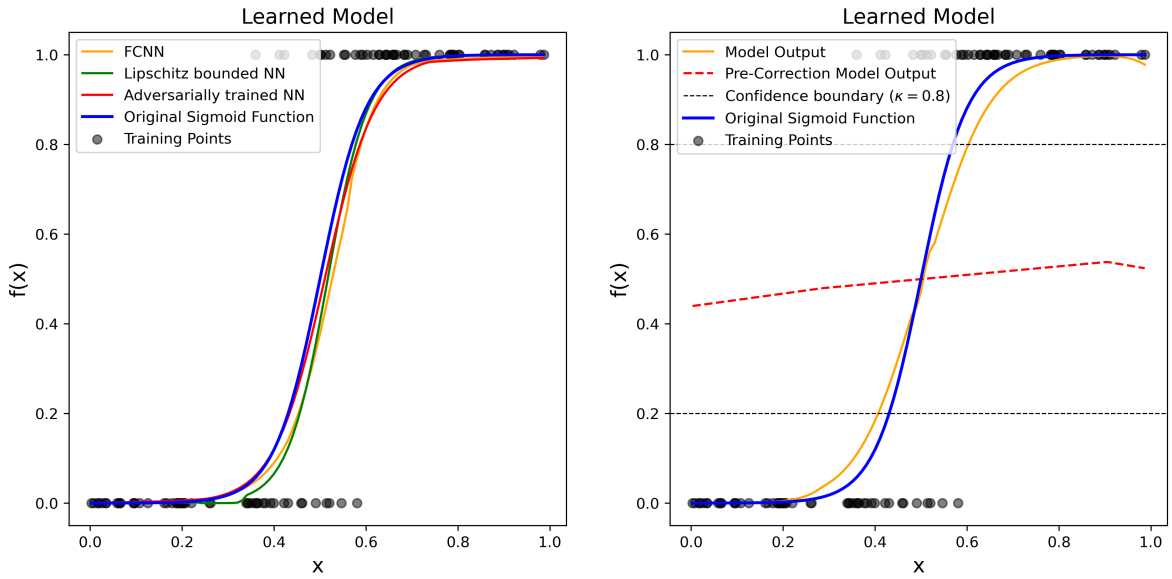
The model achieved 83.2% coverage and 92.1% average confidence, which is not far behind the other methods. Therefore, the binary cross-entropy loss function might sufficiently prioritize confidence for the 0.8 confidence threshold. This contributes to research question 2.

Table 3.2: Model performance evaluation (Mean $\pm$ Standard Deviation).

Model	Accuracy	Coverage	Confidence	Loss	ER(1/255)	ER(8/255)	ER(32/255)
Lipschitz	0.927 ± 0.024	**0.884** ± 0.066	**0.941** ± 0.033	0.295 ± 0.256	0.999 ± 0.002	0.988 ± 0.020	0.863 ± 0.065
Adversarial	0.928 ± 0.024	0.735 ± 0.285	0.883 ± 0.091	0.215 ± 0.096	**1.000** ± 0.000	**1.000** ± 0.000	0.910 ± 0.052
FCNN	0.943 ± 0.010	0.850 ± 0.023	0.921 ± 0.022	0.154 ± 0.028	**1.000** ± 0.000	**1.000** ± 0.000	0.901 ± 0.025
Proposed	**0.946** ± 0.004	0.832 ± 0.034	0.921 ± 0.015	**0.149** ± 0.005	**1.000** ± 0.000	**1.000** ± 0.000	**0.918** ± 0.035

3.2.2 Experiment 2

Experimental setup. The second set of experiments focuses on interpreting and measuring the effects of the hyperparameters of the proposed method on model performance. We investigate the parameter that controls the Lipschitz constant bound in unconfident regions δ, the parameter that controls the Lipschitz constant bound in confident regions c, and the Lipschitz



(a) Fitted fully connected, Lipschitz-bounded, and adversarially trained neural network functions.

(b) Fitted proposed neural network function. The initial logit network (g_θ) is illustrated as the pre-correction model output. Confidence boundary lines are drawn to illustrate robust regions.

Figure 3.2: Comparison of model outputs: (a) Fully connected, adversarially trained, and Lipschitz constrained networks, (b) proposed method

constant of the initial logit network l_g. For interpretability, the experiment is conducted using the synthetic sigmoid dataset with slope 20.

Training setup. All other hyperparameters are fixed to the best values found in the experiment discussed in section 3.2.1 for this dataset. For example, when studying the effect of the hyperparameter l_g, other hyperparameters, such as the learning rate and c, are selected based on the best-performing model from the previous experiment.

For each hyperparameter value, three independent models are trained for 100 epochs, and the mean metrics are reported to ensure convergence, stable evaluation, and account for some stochasticity. In addition, plots of the fitted function and Lipschitz constant are provided to illustrate the effects of hyperparameter choices.

The grids explored for each parameter are:

- δ: $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}\}$
- c: $\{0, 0.1\kappa, 0.2\kappa, \ldots, 0.9\kappa, 0.999\kappa\}$
- l_g: $\{10^{-3}, 10^{-2}, 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3}, 10^{4}\}$

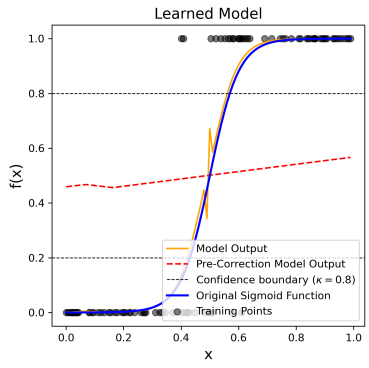
Where κ is the logit-confidence threshold.

Results. The optimal values found for each hyperparameter independently are $\delta = 10^{-1}$, $c = 0.4\kappa$, and $l_g = 10^{0}$. Now, we proceed by analyzing each hyperparameter individually.

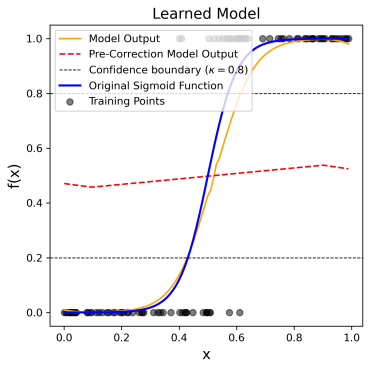
First, consider the hyperparameter δ. This hyperparameter only affects the unconfident regions of the model by bounding the Lipschitz constant at the decision boundary. This can be seen by considering that $0 < \lambda_\theta(0) = \frac{\kappa}{\delta} n_\theta(0) \leq \frac{\kappa}{\delta}$, where the bound is exact when $n_\theta(0) = 1$. δ does not affect the confidence threshold for the initial logits ($y_0 = l_g \varepsilon \frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g \varepsilon$), therefore, the subdomain of λ_θ, where δ has influence (a band $(-y_0, y_0)$ around the decision boundary) remains fixed across experiments. This is important to note because the total variation that can be present in the unconfident region also depends on the region size, which remains fixed when varying δ. Thus, the relationship between expressiveness and δ can be studied directly. Larger values of δ limit the rate of change at the decision boundary, while smaller values of δ should make the model cross the boundary swiftly. Figure 3.3 summarizes qualitatively the functions that have been fit at varying levels of delta. An illustration of the effects that changing δ has on the Lipschitz Multiplier function λ_θ is also provided. As expected, the rate of change at the decision boundary decreases as δ increases. For smaller values of δ (such as 10^{-4}), the network struggles to discard its high expressive potential, and hence fits a region of very high change around the decision boundary as seen in Figure 3.3a. The rate of change keeps decreasing as δ increases. For larger values, such as 10^3 (Figure 3.3c), it causes a very slow crossing of the decision boundary. A value of $\delta = 10^{-2}$ is shown to be a middle-ground value, which is neither too large or small, providing a smooth and (qualitatively) more optimal transition of the decision boundary (see Figure 3.3b).

However, these qualitative results are not supported quantitatively as presented in Figure 3.4b. Namely, none of the metrics have a statistically significant optimum because all standard error bands overlap. This may be due to the band around the decision boundary remaining of fixed size, therefore, allowing the model to escape the region where δ has influence quickly. In addition, the speed at which the model can escape this region remains fixed as it only depends on the Lipschitz constant of $g_\theta(x)$. Therefore, we conclude that the model is not very sensitive to the hyperparameter δ.

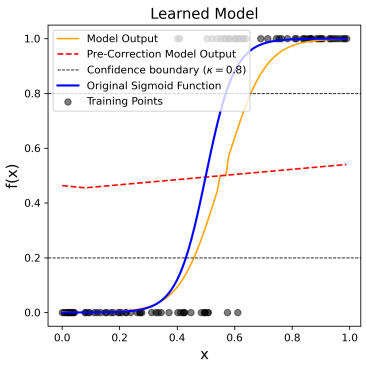
Second, consider the Lipschitz constant hyperparameter (l_g), which defines the global Lipschitz constant of the initial logit network, $g_\theta(\vec{x})$. This hyperparameter is involved in defining the logit-confidence threshold $y_0 = l_g \varepsilon \frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g \varepsilon$. I.e., the threshold that the initial logit ($g_\theta(\vec{x})$) must cross to exit the unconfident region. Furthermore, it also plays a part in defining the actual Lipschitz multiplier in confident regions: $\lambda_\theta(y) = \frac{\kappa}{l_g \varepsilon} + \frac{c-\kappa}{|y|+2 l_g \varepsilon}$. Namely, the Lipschitz multiplier decreases at higher values of l_g.
As such, the relationship between the initial Lipschitz constant l_g and the output function is rather complicated and non-trivial. Nevertheless, as shown in Table 3.3, it is clear that this hyperparameter has major effects on model performance. It is also very clear that the optimal value of l_g is in the scale of 10^0 based on the loss achieved. For smaller values of l_g, the Lipschitz multiplier of the confident region increases. This can lead to training instability as



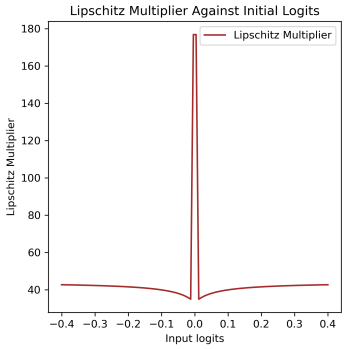
(a) Model plot at $\delta = 10^{-4}$



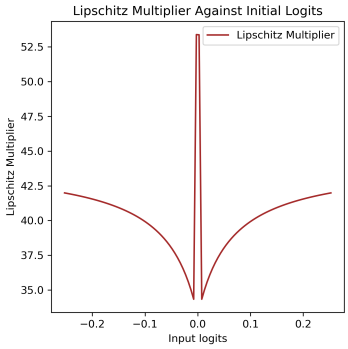
(b) Model plot at $\delta = 10^{-2}$



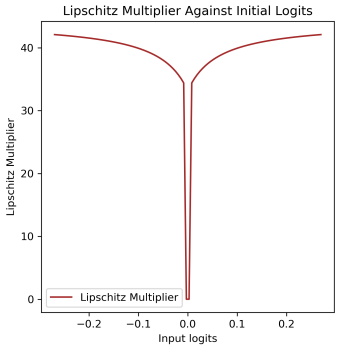
(c) Model plot at $\delta = 10^{3}$



(d) Lipschitz multiplier $\lambda_\theta(y)$ plot at $\delta = 10^{-4}$

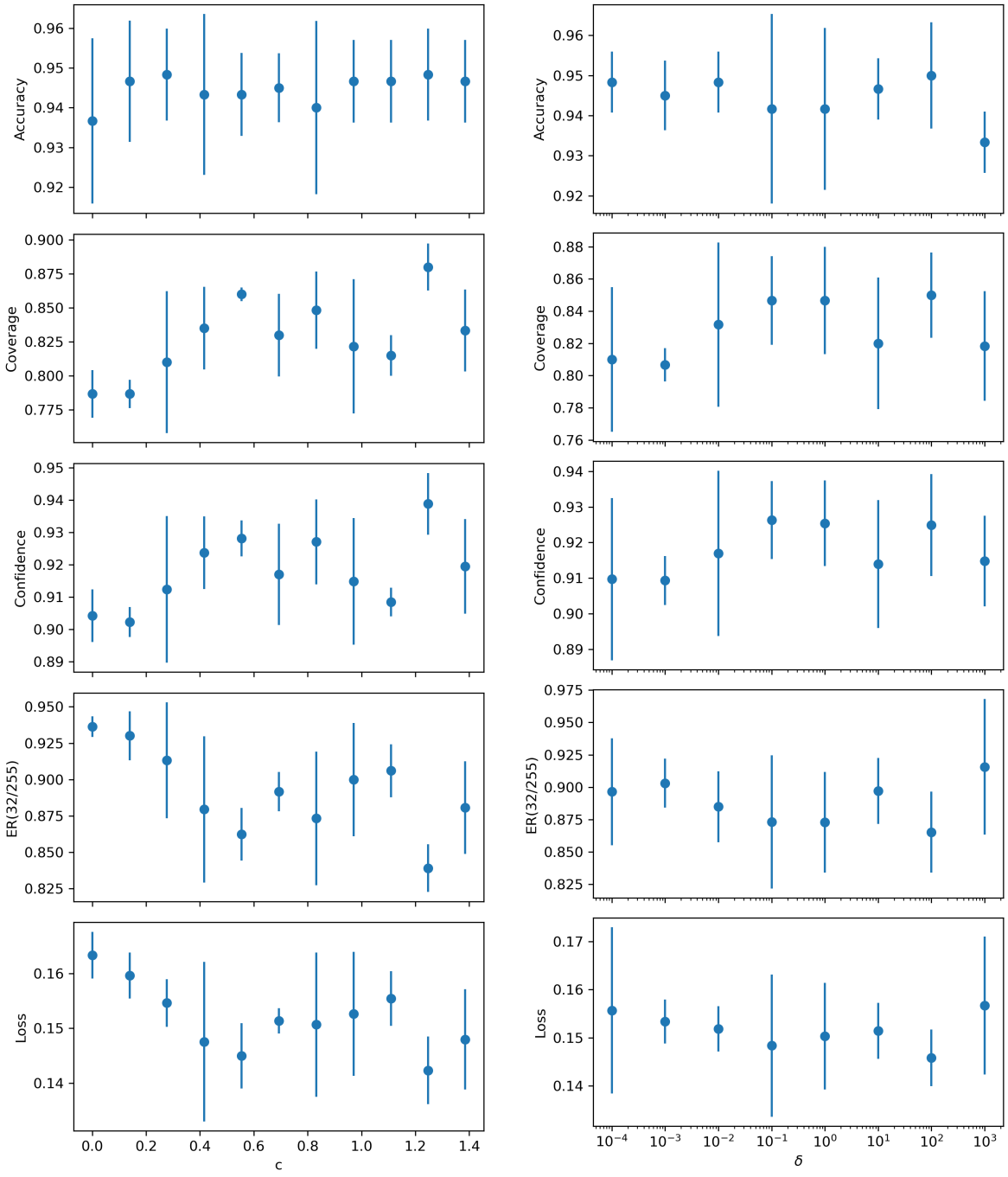


(e) Lipschitz multiplier $\lambda_\theta(y)$ plot at $\delta = 10^{-2}$



(f) Lipschitz multiplier $\lambda_\theta(y)$ plot at $\delta = 10^{3}$

Figure 3.3: Comparison of models $f_\theta(x)$ fitted at varying levels of δ, and the corresponding Lipschitz multiplier functions ($\lambda_\theta(y)$).



(a) Model performance metrics against linearly spaced values of c

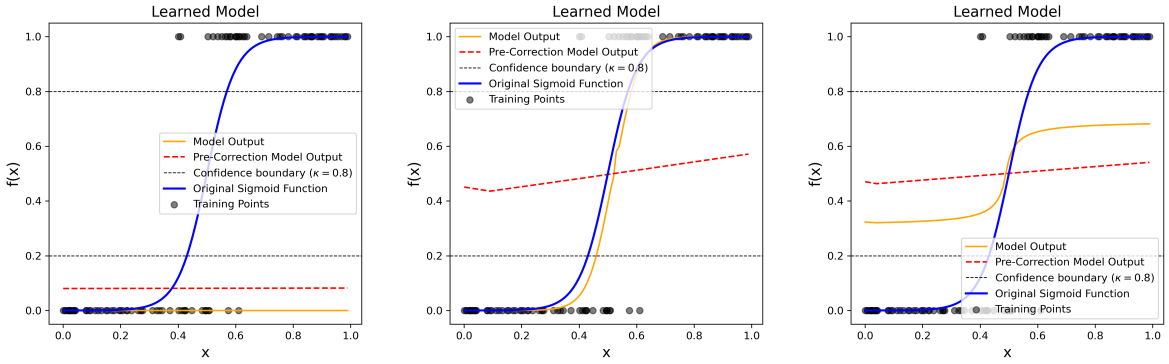
(b) Model performance metrics against logarithmically spaced values of δ

Figure 3.4: Mean accuracy, coverage, confidence, empirical robustness at $\varepsilon = 32/255$ and cross entropy loss with the standard deviation error bars aggregated from three runs against different values of hyperparameters: (a) c, and (b) δ

the model struggles to account for this large expressive power, demonstrated by the high loss in Table 3.3 and the under-fitting in Figure 3.5a.

Table 3.3: Performance across Lipschitz constant values (Mean ± Standard Deviation).

Lipschitz	Accuracy	Coverage	Confidence	Loss	EmpRob (32/255)
10^{-3}	0.532 ± 0.042	1.000 ± 0.000	1.000 ± 0.000	45682.83 ± 13307.57	1.000 ± 0.000
10^{-2}	0.575 ± 0.211	0.998 ± 0.003	0.999 ± 0.001	130.69 ± 101.59	0.978 ± 0.023
10^{-1}	**0.935** ± 0.018	0.990 ± 0.010	0.994 ± 0.002	0.988 ± 0.437	0.850 ± 0.017
10^{0}	**0.932** ± 0.024	0.838 ± 0.028	0.922 ± 0.012	**0.164** ± 0.049	0.865 ± 0.043
10^{1}	**0.925** ± 0.028	0.000 ± 0.000	0.627 ± 0.019	0.490 ± 0.023	1.000 ± 0.000
10^{2}	0.797 ± 0.248	0.000 ± 0.000	0.634 ± 0.022	0.545 ± 0.131	1.000 ± 0.000
10^{3}	0.650 ± 0.247	0.000 ± 0.000	0.646 ± 0.029	0.659 ± 0.148	1.000 ± 0.000
10^{4}	0.677 ± 0.231	0.000 ± 0.000	0.653 ± 0.029	0.641 ± 0.154	1.000 ± 0.000



(a) Model plot at $l_g = 10^{-3}$ (b) Model plot at $l_g = 1$ (c) Model plot at $l_g = 10^{2}$

Figure 3.5: Comparison of models $f_\theta(x)$ fitted at varying levels of l_g.

Coverage and confidence seem to decrease with an increase in the Lipschitz constant, which might be due to the negative correlation of the Lipschitz multiplier and l_g. The model might not prefer to leave the unconfident region because the rate of change is too limited in the confident region. This is supported by Figure 3.5c, where the model stays within the unconfident band throughout.

Finally, consider the hyperparameter c. Similarly to l_g, its effect can be observed in the confidence threshold for the Lipschitz multiplier function, $y_0 = l_g\varepsilon\frac{\sqrt{c^2+8\kappa^2}-c}{2\kappa} - l_g\varepsilon$, and in the Lipschitz multiplier function in the confident region, $\lambda_\theta(y) = \frac{\kappa}{l_g\varepsilon} + \frac{c-\kappa}{|y|+2l_g\varepsilon}$. However, the effect of this parameter is not as significant because of the small range of possible values for this parameter, $0 \leq c < \kappa$. This can be seen in Figure 3.4a, where the accuracy remains the same across different values of c. Nevertheless, there is a slight trend such that small values of c achieve higher loss, lower average confidence, coverage, but higher robustness. Lower values of c cause the Lipschitz multiplier in the confident region to be lower, in addition to

increasing the unconfident band width (y_0). Therefore, it may be more difficult for the model to leave the unconfident region as the initial network Lipschitz constant, l_g, remains fixed, but the unconfident band increases. Nonetheless, conclusions are difficult to make in this case due to the little evidence. No plots of the fitted model are presented due to the lack of variability for different values of c.

To conclude, the hyperparameters δ and c do not seem to affect the model fit in this simple sigmoid classification case. However, the hyperparameter l_g does have a significant effect. A reparametrization of the current hyperparameters to have a more intuitive and simple form might yield better interpretation. For example, $\delta_0 := \frac{\kappa}{\delta}$ as this directly defines the Lipschitz Multiplier value at the decision boundary.
Further iterations of the same experiment on different unidimensional functions would likely be more informative. In addition, expanding the δ grid to include very large and very small values could also reveal some additional insight. This concludes our investigation of research question 3.

Chapter 4

Conclusions

In this section, we make conclusions about the effectiveness of the proposed method, discuss future work, and reflect on the project requirements.

4.1 Future Work

Due to the novelty of the method, there are various areas of improvement and knowledge gaps surrounding this method. Hence, here, I propose further avenues of work:

- Ablation study on the unconfident network, n_θ. The network n_θ enables the unconfident network not to fully use up the Lipschitz bound that is provided to it by λ_θ. We hypothesize that n_θ is crucial to give the network enough flexibility to be practical.

- Empirical evaluation on other datasets. The current evaluation is very limited as only one simple $\mathbb{R} \to \mathbb{R}$ data-generating function was evaluated (sigmoid). To make further general statements about the performance of this method, it is required to evaluate it on (1) additional, simple data-generating functions (tanh, sin), or (2) data with multi-dimensional inputs, such as CIFAR-10.

- Empirical evaluation of applicability to different architectures. By design, the proposed method is architecture-agnostic. The method only requires that the Lipschitz constant of g_θ is known and bounded. Other architectures, such as CNN, have Lipschitz-bounded counterparts [PBBL24], therefore, would serve as a great avenue for future work to verify the applicability of this method.

- Extension of the method to multi-class classification. Although binary classification finds many applications in fields such as anomaly detection, multi-class classification is a more general and, therefore, more important concept. It may be possible to ensure the SoftMax function is invertible by removing a degree of freedom from the logits

(e.g., by ensuring the final logits always sum to 0). By using the invertibility of the SoftMax and using the margin as a measure of confidence, it may be possible to adapt the proofs to the multi-classification case. This would need to be investigated further.

- Enabling additional flexibility in the confident network. Adding a network multiplier similar to n_θ to the Lipschitz multiplier in the confident region is not trivial. The multiplication of n_θ in the unconfident part of the network was shown to be viable because there is no requirement for the local Lipschitz constant in this region. However, the confident region necessitates a specific local Lipschitz constant. Therefore, extra considerations of how the multiplier affects the final local Lipschitz constant are necessary.

- Exploration of different model selection metrics based on κ. As discussed in requirement 2, and in section 3.1.3, the binary cross entropy does not capture the need for high coverage dependent on κ. It would be important to investigate whether there are situations where cross entropy fails to select a model with high coverage, and what κ-dependent metrics can be used instead.

- Effects of varying ε and κ. In this project, we only investigated models with $\varepsilon = 8/255$ and $\kappa = 0.8$. To assess the applicability of the method, it would be important to study the effects of varying these parameters. Especially, looking into edge cases, such as $\varepsilon \approx 0$, might reveal interesting properties of the method.

4.2 Reflection and Conclusion

First, we reflect on the requirements of the project.

Requirement 1. The proposed method has been proven to theoretically work with multidimensional data in section 2.3. However, the output remains limited to the binary case. The method that was proposed initially worked only with one-dimensional inputs and outputs, as it is common and often beneficial to start with a simplified version of the problem. Extending it to multi-dimensional inputs proved to be straightforward. However, a crucial aspect of the method is being able to reason about the final confidence (probability), given initial logits to know whether to assign the Lipschitz Multiplier of the confident or the unconfident region. In the binary classification case, this is done simply as the sigmoid function is invertible. However, it is typical to use a non-invertible SoftMax function in the multi-class classification case. Thus, a more thorough investigation of this adaptation was required, which I failed to do due to time constraints. However, it presents an important avenue for future work as discussed previously.

Requirement 2. Both of our experiments in section 3.2.1 and section 3.2.2 have demonstrated that under suitable hyperparameters, the model will find a high-coverage solution.

Therefore, this requirement is partially completed as it was shown for a simple case.

Requirement 3. Corollary 2.3.9.1 in section 2.3 proves that given a fixed ε and κ, the proposed method guarantees ε-robustness around κ-confidence regions. Therefore, this requirement is fully completed. In addition, more general results have been derived in this project, which may be applicable to prove the sufficiency of other formulations of λ.

Requirement 4. The proposed method is guaranteed to be robust by proof. However, it is unclear whether it has succeeded in overcoming the accuracy-robustness trade-off. Experiments in section 3.2.1 showed an improved accuracy over Lipschitz neural networks. However, the learning task was very simple. Hence, no conclusions about the behaviour in more complex settings or in the general case can be made, leaving space for future work. More complex datasets were not used due to the heavy focus that I placed on the theoretical derivations, which ended up taking more time than expected, not leaving enough time for experimentation. Thus, I consider the requirement only partially completed because this result was not demonstrated in a more complex setting.

Requirement 5. Throughout the project, there were various versions of the proposed method. However, only one was fully evaluated in this report. For example, another proposed method involved having a two-network ensemble such that one is responsible for confident solutions, and the other for unconfident ones. However, due to time restrictions, these methods were not implemented, evaluated, or proven to have mathematical guarantees. Therefore, this requirement is partially completed as one method was presented.

We can conclude that the project was only partially successful, given that most requirements were only partially completed. Nevertheless, we believe this project serves as an impactful proof of concept that robustness around high-confidence regions is achievable. We have presented a novel method and have proven that it mathematically guarantees robustness. Moreover, we have empirically shown that this method is capable of learning the simple sigmoid function. The results have shown that in this simple setting, the method is comparable to FCNNs, Adversarial training, and Lipschitz-bounded networks in terms of accuracy, coverage, and loss. However, we have been unable to make general conclusions due to the lack of complexity in the training setup and datasets. Yet, we have demonstrated that this area of research should not be neglected and could be a stepping stone to more robust and safe AI methods. Nevertheless, more work is required to ensure this method is of practical significance.

Bibliography

[ABC+24] Anagha Athavale, Ezio Bartocci, Maria Christakis, Matteo Maffei, Dejan Nickovic, and Georg Weissenbacher. Verifying global two-safety properties in neural networks with confidence. In *International Conference on Computer Aided Verification*, pages 329–351. Springer, 2024.

[ACB17] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 06–11 Aug 2017.

[ALG19] Cem Anil, James Lucas, and Roger Grosse. Sorting out Lipschitz function approximation. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 291–301. PMLR, 09–15 Jun 2019.

[BMR+17] Tom B Brown, Danilo Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. *arXiv preprint arXiv:1712.09665*, 2017.

[CBG+17] Moustapha Cisse, Piotr Bojanowski, Edouard Grave, Yann Dauphin, and Nicolas Usunier. Parseval networks: Improving robustness to adversarial examples. In Doina Precup and Yee Whye Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 854–863. PMLR, 06–11 Aug 2017.

[COJ+17] Wojciech M. Czarnecki, Simon Osindero, Max Jaderberg, Grzegorz Swirszcz, and Razvan Pascanu. Sobolev training for neural networks. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[CRK19] Jeremy Cohen, Elan Rosenfeld, and Zico Kolter. Certified adversarial robustness via randomized smoothing. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 1310–1320. PMLR, 09–15 Jun 2019.

[Don19] Minghzi Dong. *Metric Learning with Lipschitz Continuous Functions*. PhD thesis, UCL (University College London), 2019.

[EEF+18] Kevin Eykholt, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning visual classification. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1625–1634, 2018.

[GB10] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In Yee Whye Teh and Mike Titterington, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256, Chia Laguna Resort, Sardinia, Italy, 13–15 May 2010. PMLR.

[GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[GSS15] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *2015 International Conference on Learning Representations*. ICLR, 2015.

[HSW89] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[HZRS15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[LBBH98] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998.

[LHA+19] Qiyang Li, Saminul Haque, Cem Anil, James Lucas, Roger B Grosse, and Joern-Henrik Jacobsen. Preventing gradient attenuation in lipschitz constrained convolutional networks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[LRC20] Fabian Latorre, Paul Rolland, and Volkan Cevher. Lipschitz constant estimation of neural networks via sparse polynomial optimization. In *International Conference on Learning Representations*, 2020.

[MKKY18] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.

[MMS+18] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations*, 2018.

[MNG+21] Xingjun Ma, Yuhao Niu, Lin Gu, Yisen Wang, Yitian Zhao, James Bailey, and Feng Lu. Understanding adversarial attacks on deep learning based medical image analysis systems. *Pattern Recognition*, 110:107332, 2021.

[PBBL24] Bernd Prach, Fabio Brau, Giorgio Buttazzo, and Christoph H. Lampert. 1-Lipschitz Layers Compared: Memory, Speed, and Certifiable Robustness. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24574–24583, Seattle, WA, USA, June 2024. IEEE.

[PC24] European Parliament and Council. Regulation (eu) 2024/1689 on artificial intelligence (ai act), 2024. Article 15: Accuracy, Robustness and Cybersecurity.

[PL22] Bernd Prach and Christoph H. Lampert. Almost-orthogonal layers for efficient general-purpose lipschitz networks. In Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner, editors, *Computer Vision – ECCV 2022*, pages 350–365, Cham, 2022. Springer Nature Switzerland.

[Rud17] Sebastian Ruder. An overview of gradient descent optimization algorithms, 2017.

[SBBR16] Mahmood Sharif, Sruti Bhagavatula, Lujo Bauer, and Michael K. Reiter. Accessorize to a crime: Real and stealthy attacks on state-of-the-art face recognition. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and*

Communications Security, CCS '16, page 1528–1540, New York, NY, USA, 2016. Association for Computing Machinery.

[SMGS+20] Mathieu Serrurier, Franck Mamalet, Alberto González-Sanz, Thibaut Boissin, Jean-Michel Loubes, and Eustasio del Barrio. Achieving robustness in classification using optimal transport with hinge regularization, 2020.

[SZS+14] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In Yoshua Bengio and Yann LeCun, editors, *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014.

[TPV25] Khoat Than, Dat Phan, and Giang Vu. Gentle local robustness implies generalization. *Machine Learning*, 114(6):142, April 2025.

[TRG19] Simen Thys, Wiebe Van Ranst, and Toon Goedemé. Fooling automated surveillance cameras: Adversarial patches to attack person detection. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 49–55, 2019.

[TSS18] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

[XM12] Huan Xu and Shie Mannor. Robustness and generalization. *Machine Learning*, 86(3):391–423, March 2012.

[YRZ+20] Yao-Yuan Yang, Cyrus Rashtchian, Hongyang Zhang, Russ R Salakhutdinov, and Kamalika Chaudhuri. A closer look at accuracy vs. robustness. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 8588–8601. Curran Associates, Inc., 2020.

[ZCS+19] Huan Zhang, Hongge Chen, Zhao Song, Duane Boning, Inderjit Dhillon, and Cho-Jui Hsieh. The limitations of adversarial training and the blind-spot attack. In *International Conference on Learning Representations*, 2019.